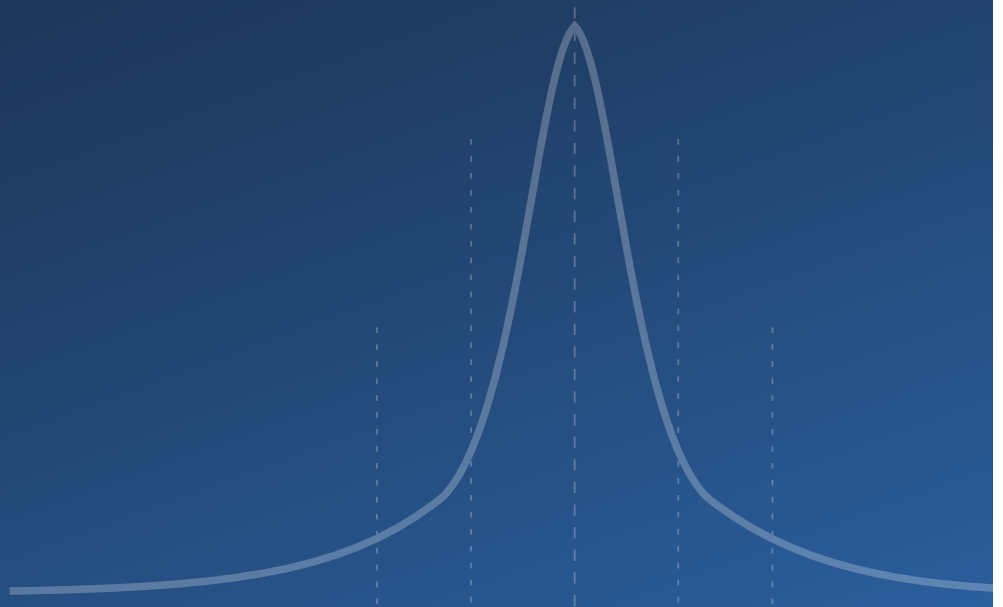


Lognormal Returns & xlwings

The complete guide: from “what is a return?” to running, auditing, and presenting a professional volatility analysis driven from Excel with Python.



Prepared for Oscar · META & ORCL, daily timeframe
Data: Excel STOCKHISTORY, Aug 31 2021 – Sep 4 2026
Built September 14, 2026

How to use this guide

This guide assumes you know nothing about statistics, Python, or xlwings. Every idea is built from the ground up, with a worked example, and then connected to the actual files on your computer and the actual numbers for META and ORCL. By the end you should be able to run the analysis, explain every number in the output, defend the method, and extend it.

If you have 30 minutes

Ch. 1, 16, 18

The short version, how to run it, what the results mean.

If you have an evening

Parts I-III

The finance, the statistics, and xlwings in depth.

To become the expert

Everything + quiz

Do the hands-on exercises and Appendix D.

Conventions

KEY IDEA

The one sentence to remember from a section.

WATCH OUT

A common mistake or a limitation.

IN PLAIN ENGLISH

A jargon-free restatement of something technical.

DEFINITION

Precise meaning of a term (all collected in Appendix C).

TRY IT

A hands-on exercise you can do right now.

Code and file names look like `run_analysis.py`. Excel previews are drawn cell by cell from the real workbooks.

Contents

- 1 The 10-minute version

PART I – FINANCE & STATISTICS FROM ZERO

- 2 Stock prices and bars
- 3 Returns: simple, compounded, logarithmic
- 4 Describing a pile of numbers
- 5 The normal distribution and the 68-95-99.7 rule
- 6 The lognormal model of prices
- 7 Time: the square-root rule
- 8 When reality disagrees: fat tails

PART II – WHY THIS ANALYSIS MATTERS

- 9 How professionals use it
- 10 Limits and honest caveats

PART III – THE TOOLS FROM ZERO

- 11 The Excel side
- 12 Python from zero
- 13 xlwings, in depth
- 14 The code, file by file

PART IV – OPERATING MANUAL

- 15 Installation and first run
- 16 Everyday use
- 17 Tour of the output workbook
- 18 Reading and presenting the results
- 19 Troubleshooting

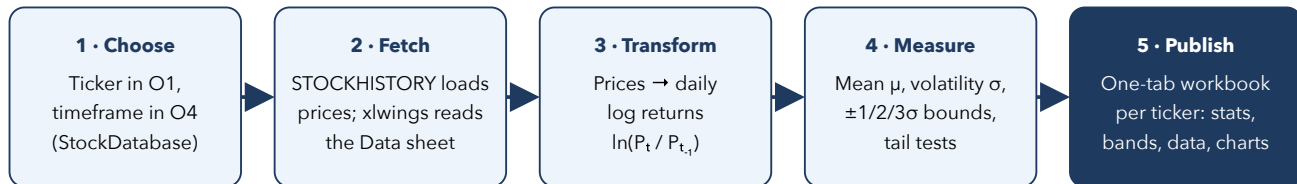
APPENDICES

- A Formula sheet
- B Excel ↔ Python mapping
- C Glossary
- D Self-test quiz with answers
- E xlwings cheat sheet

The 10-minute version

The assignment asks one practical question: **how far does this stock usually move in a day, and how often does it move much further?** We answer it with the standard toolkit of quantitative finance, using Excel for the data and presentation and Python for the computation.

What we did, in five steps



Excel at both ends (data in, results out) · Python in the middle (the math) · xlwings is the bridge

The headline numbers

META typical day (1σ)

±2.85%

≈ 45% annualized volatility

ORCL typical day (1σ)

±2.69%

≈ 43% annualized volatility

Days inside $\pm 1\sigma$ (theory 68%)

82% · 81%

META · ORCL: more quiet days than theory predicts

Next-day range from last close	Probability (normal)	META (\$616.77)	ORCL (\$158.78)
$\pm 1\sigma$	68.27%	\$599.67 – \$634.85 (–2.77% / +2.93%)	\$154.64 – \$163.18 (–2.61% / +2.77%)
$\pm 2\sigma$	95.45%	\$582.82 – \$653.21 (–5.51% / +5.91%)	\$150.54 – \$167.62 (–5.19% / +5.57%)
$\pm 3\sigma$	99.73%	\$566.44 – \$672.09 (–8.16% / +8.97%)	\$146.55 – \$172.19 (–7.70% / +8.44%)
Days actually beyond $\pm 3\sigma$ (1,258 days)	≈ 3.4 expected	15 days	20 days

What it means

- **Both stocks move a lot.** About 2–3 times the day-to-day movement of the overall S&P 500. A “normal” META day is $\pm \$17$ per share.
- **The textbook bell curve is only a first approximation.** Most days are calmer than it predicts, and a few days are wildly more violent. Statisticians call this *fat tails*, and it is the most important finding.
- **The extreme days are earnings announcements.** META fell 26% on Feb 3, 2022 (a 10.8σ move, which a normal distribution says should essentially never happen). ORCL rose 36% on Sep 10, 2025.

- **Practical use:** $\pm 2\sigma$ is a sensible “normal day” range for setting expectations, stops, and position sizes. It is *not* a worst case. Treat earnings days separately.

KEY IDEA

Volatility (σ) tells you the size of a typical move. The shape of the distribution (skew and fat tails) tells you how badly σ understates the rare, extreme moves. A professional analysis reports both.

PART I

Finance & statistics from zero

Everything needed to understand the numbers: what a price bar is, why we use log returns, what standard deviation measures, where 68-95-99.7 comes from, and how to tell when the model is wrong.

| Chapters 2-8 · Worked examples with six prices you can check on a calculator · Real META and ORCL figures

Stock prices and bars

Before any statistics, we need to know exactly what data we have.

2.1 A price is the last trade

A share of META is a small slice of ownership in Meta Platforms. Its **price** is whatever the last buyer and seller agreed on. Prices change thousands of times per second while the market is open (9:30 am – 4:00 pm New York time). To analyse them we summarise each period into a **bar**.

2.2 OHLCV: the five numbers in a bar

Field	Meaning	Used in our analysis?
Open	First trade of the period	Kept for reference
High	Highest trade of the period	Kept for reference
Low	Lowest trade of the period	Kept for reference
Close	Last trade of the period (the “official” price)	Yes: all returns are close-to-close
Volume	Number of shares traded	Kept for reference

Here is what the data looks like in `StockDatabase.xlsx` (currently showing IBM, which was the ticker saved in the file):

StockDatabase.xlsx							Data
A1	=STOCKHISTORY(01,02,03,VLOOKUP(04,R1:S3,2,FALSE),1,0,2,3,4,1,5)						
	A	B	C	D	E	F	
1	Date	Open	High	Low	Close	Volume	
2	8/31/21	\$139.54	\$140.94	\$138.95	\$140.34	4235101	
3	9/1/21	\$139.98	\$140.07	\$139.19	\$139.30	2474544	
4	9/2/21	\$139.72	\$140.05	\$139.03	\$140.01	2715659	
5	9/3/21	\$139.68	\$140.47	\$139.30	\$139.58	1924215	
6	9/7/21	\$139.65	\$139.79	\$137.76	\$138.06	3285363	
7	9/8/21	\$138.14	\$139.09	\$137.60	\$138.67	2985409	
8	9/9/21	\$137.85	\$138.96	\$137.56	\$137.74	3508363	

StockDatabase.xlsx → Data sheet, columns A-F: the table STOCKHISTORY spills from cell A1.

2.3 Timeframes: daily, weekly, monthly

A **daily bar** summarises one trading day, a **weekly bar** one week, a **monthly bar** one month. The “temporality” you choose changes both the number of observations and what a “move” means:

Timeframe	History pulled by StockDatabase	≈ Bars	Bars per year	Answers the question...
DAILY	≈ 5 years	1,260	252	How much can it move tomorrow?
WEEKLY	≈ 10 years	520	52	How much over a week? (swing trades)
MONTHLY	≈ 20 years	240	12	How much over a month? (position investing)

DEFINITION • 252

The number of trading days in a typical US year: 365 days minus 104 weekend days minus about 9 market holidays.

2.4 Adjusted vs unadjusted prices

When a company does a **stock split** (e.g. 1 share becomes 10), the price drops mechanically by 90% without anyone losing money. Data providers *adjust* history for splits so such fake “crashes” don’t appear. Excel’s STOCKHISTORY is split-adjusted. **Dividends** are another matter: on the ex-dividend date the price drops by the dividend amount. STOCKHISTORY does *not* add dividends back, so our returns are *price returns*, not *total returns*. For META and ORCL (small dividends) the effect on daily volatility is negligible.

Returns: simple, compounded, logarithmic

We never analyse prices directly. We analyse **returns**, the percentage change from one bar to the next. This chapter explains why, and why we specifically use *log* returns.

3.1 Why returns and not prices?

- **Comparability.** A \$5 move means 5% for a \$100 stock and 0.6% for a \$790 stock. Percentages put META (\$617) and ORCL (\$159) on the same scale.
- **Stability.** Prices trend and wander (META went from \$379 to \$89 to \$790). The distribution of prices over five years tells you nothing about tomorrow. Returns, by contrast, fluctuate around a stable centre near zero, so their past behaviour is a reasonable guide to their future behaviour. Statisticians call this *stationarity*.

3.2 Simple return

$$R_t = P_t / P_{t-1} - 1$$

"today's close divided by yesterday's close, minus one"

If META closes at \$600 and then \$612, $R = 612/600 - 1 = 0.02 = +2\%$. This is the number you see on a brokerage app.

3.3 The problem: simple returns don't add up

Suppose a stock rises 10% and then falls 10%. Adding the returns suggests you are back to 0%. In reality: \$100 → \$110 → \$99. You lost 1%. Simple returns **compound** (multiply): $(1.10)(0.90) - 1 = -1\%$. Multiplication is awkward for statistics, which is built on sums and averages.

3.4 A two-minute refresher on e and ln

IN PLAIN ENGLISH

$e \approx 2.71828$ is the number you get from compounding continuously: \$1 at 100% interest compounded every instant grows to \$2.718 in a year.

$\ln(x)$, the natural logarithm, is the reverse question: "to what power must I raise e to get x?" So $\ln(e) = 1$, $\ln(1) = 0$, $\ln(2) \approx 0.693$.

The magic property: $\ln(a \times b) = \ln(a) + \ln(b)$. Logarithms turn multiplication into addition. That is exactly what we need to fix the compounding problem.

3.5 Log return

$$r_t = \ln(P_t / P_{t-1}) = \ln(1 + R_t)$$

In Excel: =LN(E3/E2) · in Python: np.log(close / close.shift(1))

Going the other way: $P_t = P_{t-1} \cdot e^{r_t}$, and $R = e^r - 1$.

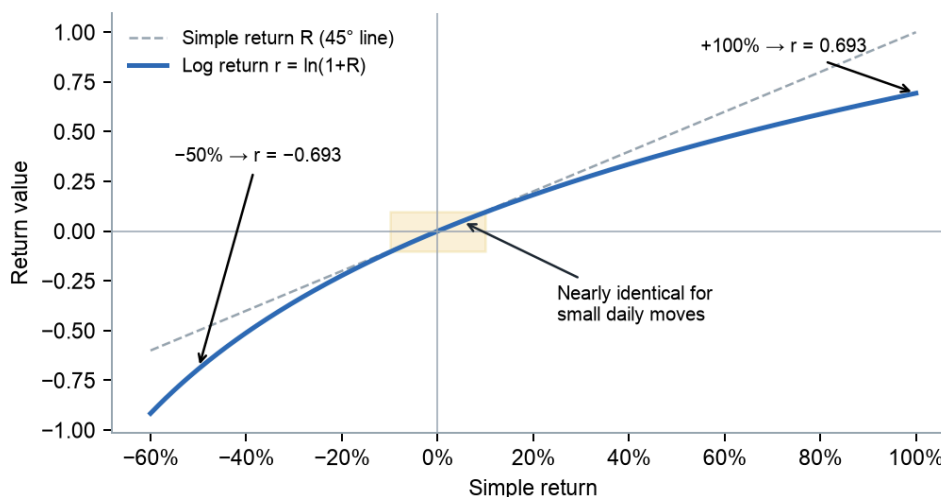


Figure 3.1 For everyday moves (shaded) the two are almost identical. They diverge for big moves: a +100% gain is $r = +0.693$ and a -50% loss is $r = -0.693$. Both cancel exactly, because $\$100 \rightarrow \$200 \rightarrow \$100$.

3.6 Worked example: six prices

Take six daily closes: 100, 102, 99, 101, 104, 103. Grab a calculator and follow along.

Day	Close P	P / previous	Simple return R	Log return $r = \ln(P/\text{prev})$
0	100	—	—	—
1	102	1.020000	+2.000%	+0.019803
2	99	0.970588	-2.941%	-0.029853
3	101	1.020202	+2.020%	+0.020001
4	104	1.029703	+2.970%	+0.029270
5	103	0.990385	-0.962%	-0.009662
Sum			+3.087% ✗	+0.029559 ✓

The true 5-day change is $103/100 = +3\%$, and $\ln(1.03) = 0.029559$, *exactly* the sum of the log returns. The simple returns add to 3.087%, which is wrong. This additivity is the core reason for log returns.

3.7 The four reasons we use log returns

Reason	Why it matters for this analysis
1. Time-additive	A week's log return is the sum of five daily ones. Sums of random numbers are what the square-root-of-time rule (Ch. 7) and the Central Limit Theorem are built on.
2. Symmetric	+x and -x cancel exactly, so up and down moves are treated consistently.
3. Prices stay positive	Any log return, however negative, gives $P = P_0 \cdot e^r > 0$. A normal model on simple returns would allow impossible -150% losses.
4. Industry standard	Black-Scholes option pricing, geometric Brownian motion, and the volatility quoted by every options desk assume normally distributed log returns. Your numbers are directly comparable to theirs.

Describing a pile of numbers

After step 3 we have 1,258 daily log returns per stock. Nobody can look at 1,258 numbers, so we summarise them: where is the centre, how spread out are they, and what shape do they make?

4.1 The histogram: seeing the shape

Divide the range of returns into equal-width buckets (“bins”), count how many days fall in each, and draw a bar per bin. The result is a **histogram**, a picture of the **distribution**. Every analysis tab contains this chart.

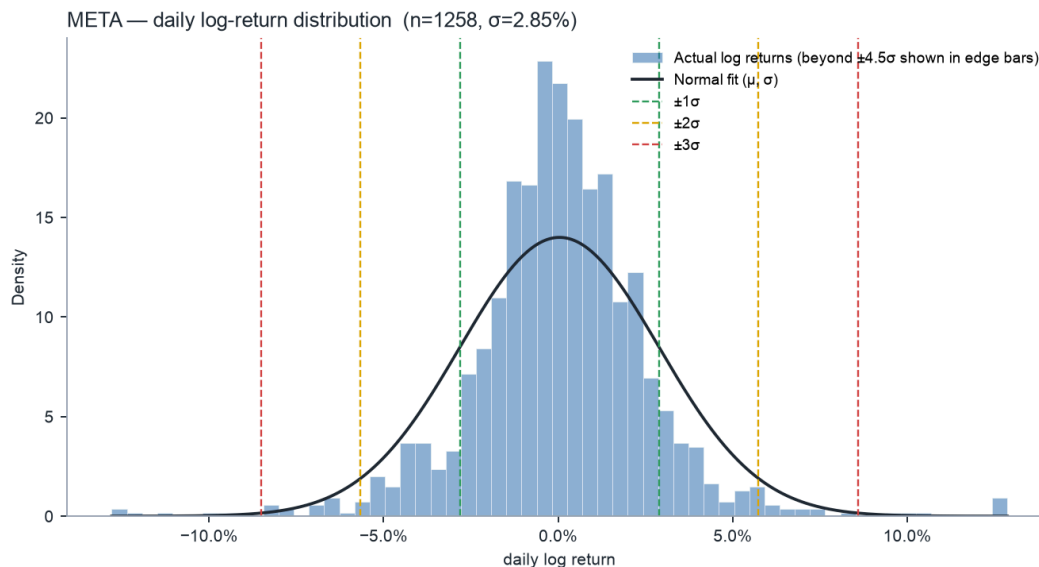


Figure 4.1 META: 1,258 daily log returns. Bars are the real data; the black curve is the normal distribution with the same mean and standard deviation; the dashed lines are $\pm 1\sigma$, $\pm 2\sigma$, $\pm 3\sigma$.

4.2 The mean μ (the centre)

$$\mu = (r_1 + r_2 + \dots + r_n) / n$$

Excel =AVERAGE(range) · Python returns.mean()

Worked example: $(0.019803 - 0.029853 + 0.020001 + 0.029270 - 0.009662) / 5 = 0.029559 / 5 = 0.005912$, i.e. +0.59% per day.

4.3 Variance and standard deviation σ (the spread)

How far, typically, is each return from the mean? We can't just average the distances, because positive and negative deviations cancel. So we square them (making everything positive), average, and take the square root to get back to the original units.

$$\sigma = \sqrt{[\sum (r_i - \mu)^2 / (n - 1)]}$$

Excel =STDEV(range) (same as STDEV.S) · Python returns.std(ddof=1)

Day	r	r - μ	(r - μ) ²
1	0.019803	+0.013891	0.000193
2	-0.029853	-0.035765	0.001279
3	0.020001	+0.014089	0.000198
4	0.029270	+0.023359	0.000546
5	-0.009662	-0.015574	0.000243
Sum of squares 0.002459 · divide by (n - 1) = 4 → variance 0.000615 · square root →			σ = 0.024792 = 2.48%

DEFINITION · VOLATILITY

In finance, **volatility** simply means the standard deviation of returns. “META has 45% volatility” means σ of its annual log returns is about 45%.

WATCH OUT · WHY N - 1?

We are estimating the spread of *all possible* returns from a *sample*. Because the sample mean is fitted to the same data, deviations from it are slightly too small; dividing by n - 1 instead of n corrects this bias (Bessel’s correction). With 1,258 observations the difference is 0.04%, but it is the correct method and what Excel’s STDEV does.

4.4 Median, minimum, maximum

The **median** is the middle value when sorted; unlike the mean it ignores how extreme the extremes are. **Min** and **max** are the worst and best days. For META: median +0.08%, worst -30.6% (log), best +20.9% (log).

4.5 Why the mean matters less than σ for daily data

META’s daily μ is 0.039%; its σ is 2.85%, 74 times larger. The uncertainty in our estimate of μ (its *standard error*, $\sigma/\sqrt{n} = 2.85\%/\sqrt{1258} = 0.08\%$) is **bigger than μ itself**. In plain English: five years of data cannot tell us reliably whether META’s true average daily drift is positive. But σ is estimated very precisely. That is why this analysis is about **ranges**, not predictions of direction.

The normal distribution and the 68-95-99.7 rule

The assignment asks for 68%, 95% and 99% bounds, “a 1, 2 and 3 sigma move.” Those percentages come from one specific shape: the normal distribution.

5.1 The bell curve

The **normal** (or Gaussian) distribution is the symmetric bell shape that appears whenever many small independent effects add up. That is the Central Limit Theorem, one of the deepest results in mathematics. A day’s return is the sum of thousands of trades and news items, which is the classical argument for why returns might be roughly normal. It is fully described by just two numbers: the centre μ and the spread σ .

The 68-95-99.7 rule for a normal distribution

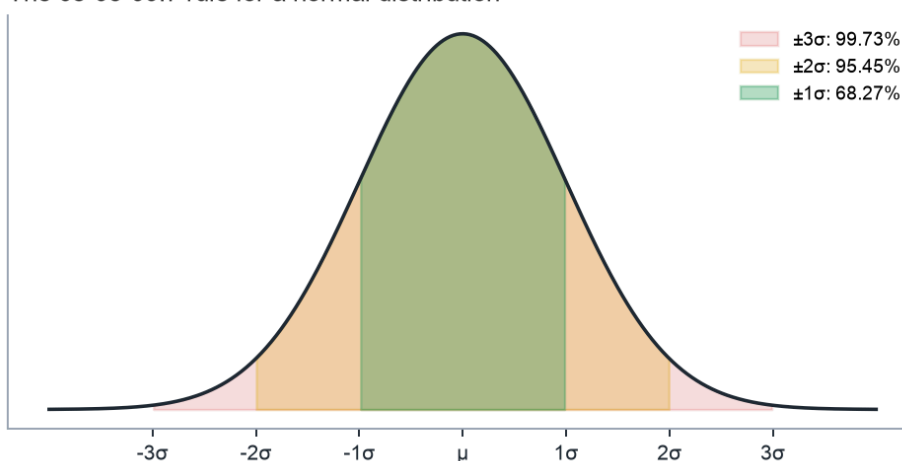


Figure 5.1 The share of a normal distribution within 1, 2 and 3 standard deviations of the mean.

5.2 The z-score: measuring moves in σ units

$$z = (r - \mu) / \sigma$$

“how many standard deviations away from average was this day?”

META’s σ is 2.85%, so a -5.7% day has $z \approx -2$: a “two-sigma down day.” Converting to z-scores makes stocks with different volatilities comparable: a 3σ day is equally unusual for a calm utility and for META. Column L of the analysis tab has the z-score of every day.

5.3 The exact percentages

Band	Probability inside	Probability outside	Outside, both tails: one day in...	Expected days outside of 1,258
$\mu \pm 1\sigma$	68.27%	31.73%	3.2 days	399
$\mu \pm 2\sigma$	95.45%	4.55%	22 days ($\approx$ monthly)	57
$\mu \pm 3\sigma$	99.73%	0.27%	370 days ($\approx$ 1.5 years)	3.4

In Excel, the probability inside $\pm k \sigma$ is `=2*NORMSDIST(k)-1`. NORMSDIST is the cumulative normal: the probability of a value below k.

WATCH OUT · "99%" VS 3σ

The rule is often rounded to "68-95-99," but 3σ is really 99.73%. An exact 99% band is $\pm 2.576\sigma$ and an exact 95% band is $\pm 1.960\sigma$. Our tables use the sigma multiples the assignment asked for and label the true probability next to each.

5.4 Two ways to use a band

- **Forward-looking:** "If returns are normal with this μ and σ , tomorrow's close lands in this range with 68% probability." That is the price-range table.
- **Backward-looking test:** "Did 68% of past days actually land inside $\pm 1\sigma$?" That is the empirical coverage column. If not, the normal model is wrong in a measurable way (Chapter 8).

The lognormal model of prices

The assignment says “build a distribution of the data using lognormal returns.” Precisely: we assume **log returns are normal**, which makes **prices lognormal**.

6.1 The model in one line

$$r_t \sim N(\mu, \sigma^2), \text{ independent each day} \implies P_T = P_0 \cdot e^{r_1 + r_2 + \dots + r_T}$$

This is the discrete-time version of Geometric Brownian Motion, the model behind Black-Scholes.

A variable is **lognormal** if its logarithm is normal. Since $\ln(P_T/P_0)$ is a sum of normal log returns, it is normal, and so the price itself is lognormal.

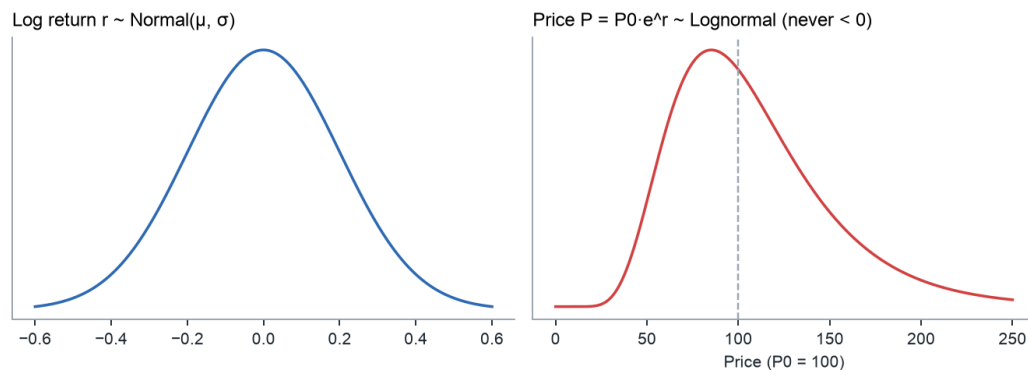


Figure 6.1 Symmetric in log-return space (left), skewed in price space (right). Prices can double but can't go below zero.

6.2 From a log band to a price band: step by step

Using the worked example ($\mu = 0.005912$, $\sigma = 0.024792$, last close $P_0 = 103$):

- 1 **Band in log space.** 1σ : $\mu - \sigma = -0.018880$ and $\mu + \sigma = +0.030704$.
- 2 **Convert to % move** with $e^x - 1$: $e^{-0.018880} - 1 = -1.87\%$ and $e^{0.030704} - 1 = +3.12\%$.
- 3 **Convert to price** with $P_0 \cdot e^x$: $103 \times 0.98130 = \$101.07$ and $103 \times 1.03118 = \$106.21$.
- 4 **2σ the same way:** $103 \cdot e^{0.005912 \mp 0.049584} = \$98.60 - \$108.88$.

Now with META's real numbers ($\mu = 0.0386\%$, $\sigma = 2.8505\%$, $P_0 = \$616.77$):

G20 fx 81.96%

	A	B	C	D	E	F	G
19	Band	Normal confidence	Lower % move	Upper % move	Lower price	Upper price	Empirical coverage
20	$\pm 1\sigma$	68.27%	-2.77%	2.93%	\$599.67	\$634.85	81.96%
21	$\pm 2\sigma$	95.45%	-5.51%	5.91%	\$582.82	\$653.21	96.34%
22	$\pm 3\sigma$	99.73%	-8.16%	8.97%	\$566.44	\$672.09	98.81%

The $\pm 1\sigma$ / $\pm 2\sigma$ / $\pm 3\sigma$ table on the META analysis tab (A19:G22).

6.3 Why the percentages are asymmetric

META's 3σ band is -8.16% / $+8.97\%$, not $\pm 8.5\%$. In log space the band is symmetric ($\mu \pm 8.55\%$); exponentiating stretches the upside and compresses the downside. This is correct behaviour, not a bug: a lognormal price can rise without limit but can only fall 100%.

DEFINITION • MEDIAN VS MEAN PRICE

$P_0 \cdot e^{\mu h}$ is the **median** future price (50% chance above, 50% below). The **mean** future price is higher, $P_0 \cdot e^{(\mu + \sigma^2/2)h}$, because a few very large upside outcomes pull the average up. This $\sigma^2/2$ term (the “volatility drag” or Itô correction) is why a volatile stock’s average simple return exceeds its average log return.

Time: the square-root rule

How do you turn a daily volatility into an annual one, or into a one-month price range? Because log returns add, the answer is surprisingly simple.

7.1 Means add, variances add, standard deviations don't

Over h days the log return is the sum of h daily returns. If they are independent:

$$\text{mean over } h \text{ days} = \mu \cdot h \quad \text{variance over } h \text{ days} = \sigma^2 \cdot h \quad \sigma \text{ over } h \text{ days} = \sigma \cdot \sqrt{h}$$

IN PLAIN ENGLISH

Uncertainty does not grow in a straight line. Four days is not four times as uncertain as one day, only twice ($\sqrt{4} = 2$). Random ups and downs partly cancel over time.

7.2 Annualizing

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252} \quad \mu_{\text{annual}} = \mu_{\text{daily}} \times 252$$

weekly data: $\times \sqrt{52}$ and $\times 52$ · monthly data: $\times \sqrt{12}$ and $\times 12$

META: $2.8505\% \times \sqrt{252} = 2.8505\% \times 15.875 = 45.25\%$. Worked example: $2.479\% \times 15.875 = 39.36\%$.

Annualizing lets you compare a daily run with a weekly run, and your numbers with an option's implied volatility, which is always quoted annualized.

7.3 Price ranges over any horizon

$$P_0 \cdot \exp(\mu h \pm k \cdot \sigma \cdot \sqrt{h})$$

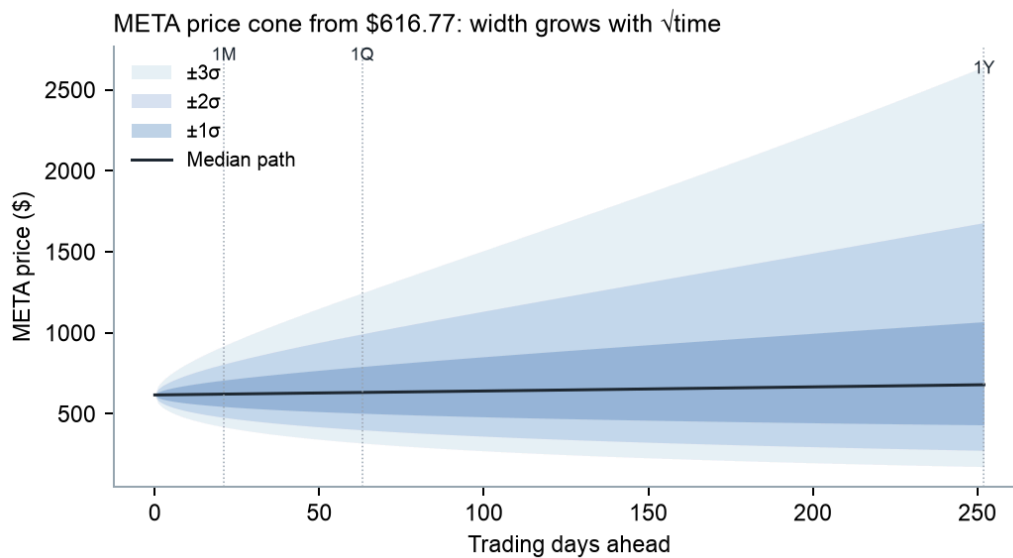


Figure 7.1 META's price cone: the range widens with the square root of time, fast at first and then more slowly.

Horizon	Bars (h)	Median price	1 σ range (68%)	2 σ range (95%)	3 σ range (99.7%)
1 day	1	\$617.01	\$599.67 - \$634.85	\$582.82 - \$653.21	\$566.44 - \$672.09
1 week	5	\$617.96	\$579.80 - \$658.63	\$544.00 - \$701.98	\$510.41 - \$748.18
1 month	21	\$621.79	\$545.65 - \$708.56	\$478.83 - \$807.44	\$420.20 - \$920.11
1 quarter	63	\$631.96	\$504.00 - \$792.42	\$401.95 - \$993.61	\$320.56 - \$1,245.88
1 year	252	\$679.83	\$432.39 - \$1,068.86	\$275.02 - \$1,680.52	\$174.92 - \$2,642.20

META, from the last close of \$616.77 with $\mu = 0.0386\%$ and $\sigma = 2.8505\%$ per day. In Excel: `=616.77*EXP( $\mu \cdot h + k \cdot \sigma \cdot \text{SQRT}(h)$ )`.

WATCH OUT · THE INDEPENDENCE ASSUMPTION

$\sqrt{t}$ scaling requires returns to be independent from day to day. For direction this is nearly true (autocorrelation of META daily returns: -0.01). But volatility clusters (Ch. 8.6), so over months the realised range can be much wider or narrower than the cone.

When reality disagrees: fat tails

A good analyst never just assumes the model is right; they test it. This is where the analysis becomes interesting, and where most of the insight lives.

8.1 Test 1: empirical coverage

Count how many real days fell inside each band and compare with theory:

Band	Normal theory	META actual	ORCL actual	Reading
$\pm 1\sigma$	68.27%	81.96% (1,031 days)	81.40% (1,024)	Far too many calm days
$\pm 2\sigma$	95.45%	96.34%	95.39%	Looks fine, but only by coincidence
$\pm 3\sigma$	99.73%	98.81% (15 outside)	98.41% (20 outside)	4–6× too many extreme days

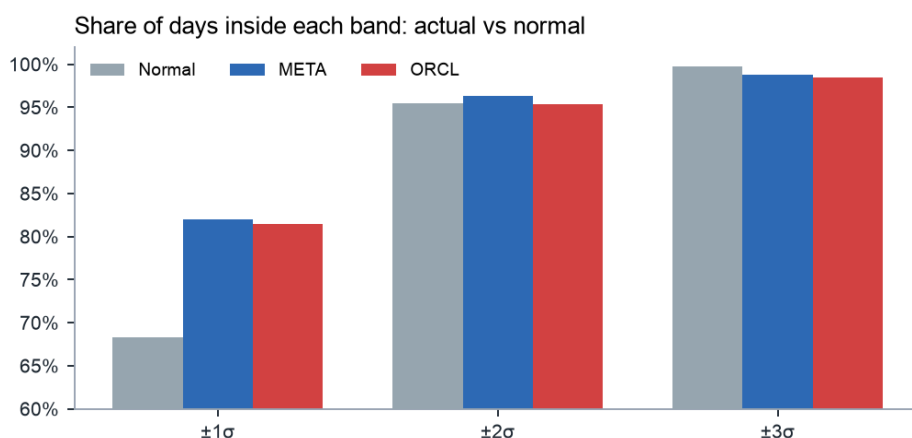


Figure 8.1 The signature of fat tails: above theory at 1σ , below theory at 3σ .

8.2 Skewness: is it lopsided?

Skew = average of z^3 (with a small-sample correction)

Excel =SKEW · 0 for a normal distribution

Cubing keeps the sign and exaggerates big values, so skew measures whether extreme moves are mostly up or mostly down. **META – 1.10**: its biggest moves were crashes (2022 earnings). **ORCL + 1.25**: dominated by its +36% jump in September 2025.

8.3 Kurtosis: how fat are the tails?

Excess kurtosis = average of z^4 – 3 (with correction)

Excel =KURT · 0 for a normal distribution

The fourth power makes kurtosis extremely sensitive to extreme observations. A normal distribution scores 3, so we subtract 3 and report *excess kurtosis*. **META 23.4, ORCL 17.9**: enormously fat-tailed. Positive excess kurtosis (“leptokurtic”) means a taller, narrower peak *and* heavier tails, exactly the pattern in Figure 8.1.

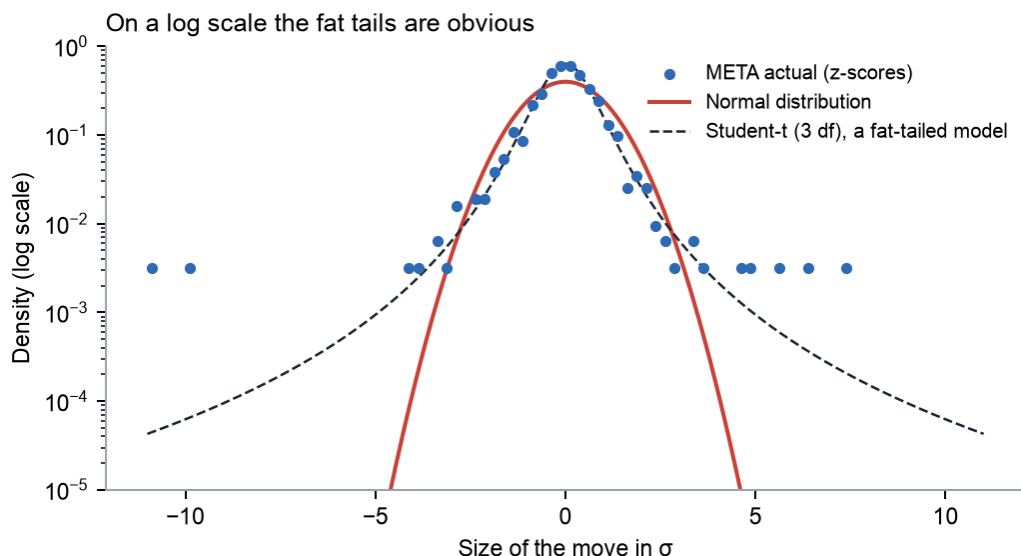


Figure 8.2 A log scale magnifies the tails. The normal curve (red) collapses toward zero beyond 4σ ; META’s real days (blue dots) keep appearing out to 10σ , closer to a fat-tailed Student-t model (dashed).

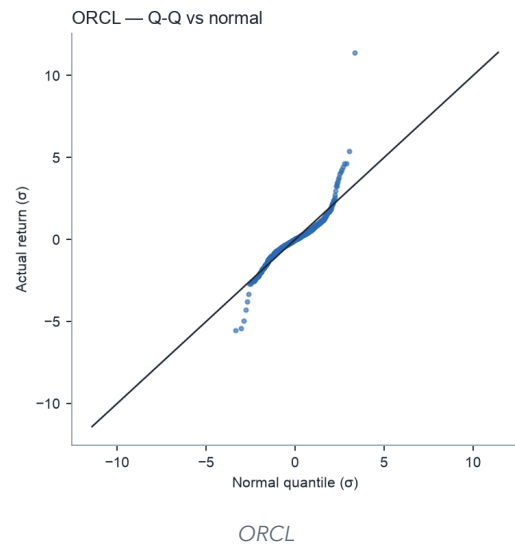
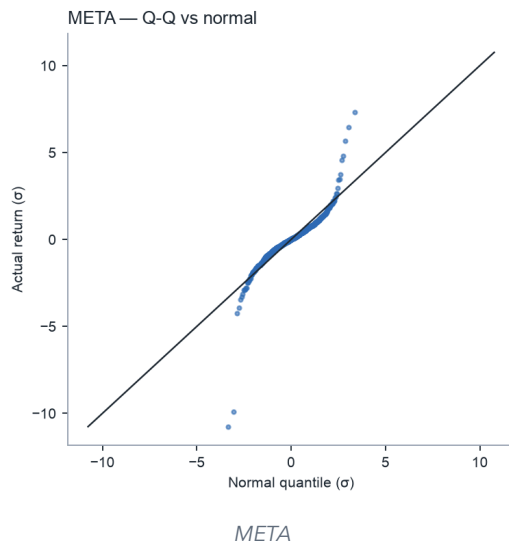
8.4 The Jarque-Bera test: a formal verdict

$$JB = n/6 \cdot (S^2 + K^2/4)$$

S = skew, K = excess kurtosis · under normality JB follows a χ^2 distribution with 2 degrees of freedom · p-value in Excel:
=CHIDIST(JB, 2)

JB is 0 for perfectly normal data and grows with skew and kurtosis. The **p-value** is the probability of seeing a JB this large *if* returns were normal. META: JB = 28,986 and ORCL: JB = 17,062, both with $p \approx 0$. Below 0.05 we **reject normality**. Any value above 5.99 already rejects at the 5% level.

8.5 The Q-Q plot: the visual test



Sort the actual z-scores, and plot each against the z-score a perfect normal sample of the same size would have at that rank. Normal data lies on the diagonal. The **S-shape** (flat in the middle, peeling away at both ends) is the fingerprint of fat tails. The furthest points are single earnings days.

8.6 Where fat tails come from

Cause 1: Event jumps

Date	Stock	Simple return	σ multiple	Event
Feb 3, 2022	META	-26.4%	-10.8 σ	Q4-2021 earnings: first-ever user decline, heavy metaverse spending
Oct 27, 2022	META	-24.6%	-9.9 σ	Q3-2022 earnings: cost guidance shock
Feb 2, 2023	META	+23.3%	+7.3 σ	Q4-2022 earnings: "year of efficiency", buyback
Feb 2, 2024	META	+20.3%	+6.5 σ	Q4-2023 earnings: first dividend
Sep 10, 2025	ORCL	+36.0%	+11.4 σ	Q1-FY26 earnings: huge cloud contract backlog
Jan 27, 2025	ORCL	-13.8%	-5.5 σ	AI-infrastructure sell-off (DeepSeek day)

Under a normal distribution a 10.8 σ day has a probability of roughly 1 in 10^{26} : it would not happen once in the age of the universe. It happened. Earnings announcements are scheduled information shocks that the "sum of many small effects" logic does not describe. Removing just the five largest days cuts META's excess kurtosis from 23.4 to about 3.5.

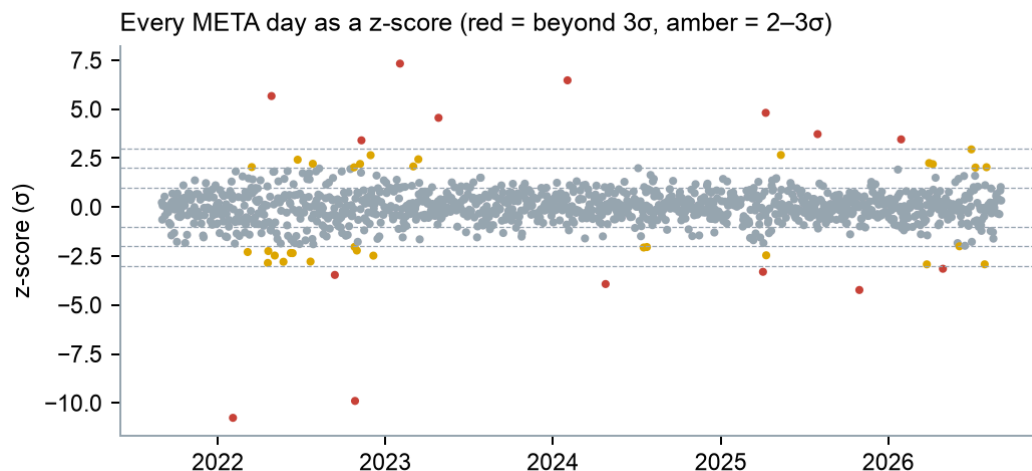


Figure 8.3 Every META trading day as a z-score. Extreme days are rare, isolated spikes, typically on earnings.

Cause 2: Volatility clustering

Volatility is not constant. Calm periods and turbulent periods alternate: "large changes tend to be followed by large changes" (Mandelbrot, 1963). Mixing a calm regime with a stormy regime, each roughly normal, produces an overall distribution that is too peaked and too fat-tailed.

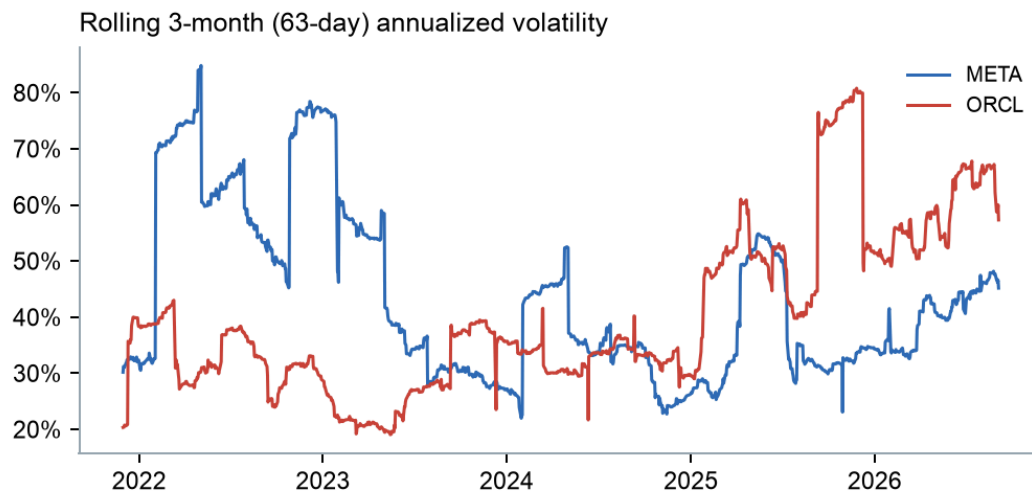


Figure 8.4 Rolling 3-month volatility. META was near 70-90% in its 2022 drawdown and near 30% in calm stretches; ORCL roughly doubled after its 2025 re-rating as an AI-infrastructure stock.

KEY IDEA

The normal model is a good description of an *ordinary* day and a poor description of an *extraordinary* one. Report σ -bands for the first, and empirical percentiles plus an event calendar for the second.

PART II

Why this analysis matters

Volatility and return distributions are the foundation of risk management, position sizing, options pricing and portfolio construction. This part connects each number in the workbook to a decision a real investor makes.

| Chapters 9-10 · Position sizing, stops, Value-at-Risk, options, earnings, portfolios · Honest limitations

How professionals use it

Nobody on a trading desk asks “what will META do tomorrow?” They ask “what range of outcomes should I be prepared for, and how big a position can I afford?” This analysis answers exactly that.

9.1 Setting expectations: what is a normal move?

If META drops 3% tomorrow, is that news? No: it’s a 1σ day, which happens about one day in three. A 9% drop (over 3σ) is news, and worth investigating. Z-scores turn a price move into a statement about how unusual it is.

9.2 Position sizing (worked example)

A common rule: never risk more than 1% of the portfolio on one idea. Suppose a \$100,000 portfolio, and you would exit META if it fell by a 2σ daily move.

- 1 Risk budget = $1\% \times \$100,000 = \$1,000$.
- 2 2σ down price = \$582.82 (from the bounds table), so risk per share = $\$616.77 - \$582.82 = \$33.95$.
- 3 Shares = $\$1,000 / \$33.95 = 29.5 \rightarrow 29 \text{ shares} \approx \$17,886$ position.
- 4 Same logic for ORCL (2σ down \$150.54, risk \$8.24/share) $\rightarrow 121 \text{ shares} \approx \$19,212$. Similar dollar risk, similar position size, because the two stocks have similar volatility.

WATCH OUT

A stop is not a guarantee. On Feb 3, 2022 META *opened* 20%+ lower; a stop at -5.5% would have filled far below its level. This is fat-tail risk in practice, and why position sizes should survive a $3\sigma+$ gap, especially through earnings.

9.3 Stop-loss placement

A stop inside the $\pm 1\sigma$ band (e.g. -2% on META) will be triggered by ordinary noise roughly one day in six (the down half of the 32% outside). Stops are better placed beyond 2σ , or scaled to σ (e.g. “ 1.5σ below entry”) so they automatically widen for volatile stocks.

9.4 Value-at-Risk (VaR)

DEFINITION • VAR

“With X% confidence, the loss over the next N days will not exceed \$Y.” It is the most widely used risk number in banking and asset management.

One-day 95% VaR for a \$10,000 META position, using the normal model (one-sided 95% = 1.645σ):

$$\text{loss} = \$10,000 \times (1 - e^{\mu - 1.645\sigma}) = \$10,000 \times (1 - e^{-0.0465}) \approx \$454$$

An empirical tail check shows where the normal model breaks: at the 0.135th percentile (the “3σ” point), META’s actual return was −16.9% versus −8.5% predicted. Normal VaR is fine at 95% and badly understates risk at 99.9%.

9.5 Options and implied volatility

Option prices contain the market’s forecast of future volatility, the **implied volatility (IV)**, quoted annualized exactly like our 45%. The market’s **expected move** over T calendar days is approximately:

$$\text{Expected } 1\sigma \text{ move} \approx S \times IV \times \sqrt{(T / 365)}$$

Example (hypothetical IV of 40%, 30 days): $\$616.77 \times 0.40 \times \sqrt{(30/365)} \approx \pm\70.7 . Compare with the realised figures: if IV is well below historical volatility, options look cheap relative to history; if well above, expensive. Around earnings IV spikes, because the market prices exactly the jump risk we found in Chapter 8.

9.6 Comparing and ranking stocks

σ puts every stock on one risk scale. META (45%) and ORCL (43%) are similar; the S&P 500 is typically 15–20%; a utility might be 20%. Returns per unit of risk (e.g. the Sharpe ratio, annual return ÷ annual volatility) only make sense once you have σ.

9.7 Portfolios and correlation

The correlation between META and ORCL daily returns is **0.27**: they often move together, but far from perfectly. A 50/50 portfolio has volatility of roughly $\sqrt{(0.25 \cdot 45^2 + 0.25 \cdot 43^2 + 2 \cdot 0.25 \cdot 0.27 \cdot 45 \cdot 43)} \approx 35\%$, lower than either stock alone. That is diversification, quantified.

9.8 Why TAMID cares

- Any investment pitch should state the risk of the idea, not just the upside. “Our target is +15% in a year” means very different things for a 20%-vol stock and a 45%-vol stock: for META, +15% sits well inside the one-year 1σ range (\$432 – \$1,069).
- It builds the core technical skill the assignment names: data from Excel → Python → computed analytics → a clean Excel deliverable. This is how analyst workflows at funds and banks actually run.

Limits and honest caveats

Knowing where a model fails is what separates an expert from someone who can run a script. Be ready to say these out loud.

Limitation	Why it matters	What an expert would do
Returns aren't normal	3σ events happen 4-6× more often than predicted (Ch. 8).	Quote empirical percentiles; fit a Student-t; use historical simulation.
Volatility isn't constant	One σ for five years mixes calm and crisis regimes.	EWMA volatility ($\lambda = 0.94$) or GARCH(1,1); rolling 1-year windows.
Event risk	Earnings produce the largest moves.	Analyse ex-earnings days separately; check the earnings calendar and options IV.
The past is not the future	ORCL's business and investor base changed in 2025; history may not represent it.	Compare with implied volatility; shorter windows.
Estimation noise in μ	Drift is not statistically distinguishable from zero.	Don't present μ as a forecast; set $\mu = 0$ for short-horizon bands.
Price, not total return	Dividends aren't included.	Use adjusted-close/total-return data for high-yield stocks.
Window choice	Results depend on which 5 years you pick.	Run WEEKLY/MONTHLY; check sensitivity.

KEY IDEA · HOW TO SAY IT IN A MEETING

"The lognormal model gives us a clean, standard baseline: META's typical daily move is about $\pm 2.9\%$, and a 2σ day is about $\pm 5.7\%$. But the data rejects normality decisively: extreme days happen four to six times more often than the model says, mostly on earnings. So we use the bands for everyday expectations and size positions to survive the tails."

PART III

The tools from zero

How Excel produces the data, what Python and its libraries are, and a complete, hands-on explanation of xlwings: what it is, how it talks to Excel, the two ways we run it, and every function in our code.

Chapters 11–14 · STOCKHISTORY decoded · Terminal and virtual environments · xlwings object model, exercises, xlwings Lite · Every function explained

The Excel side

11.1 Cells, references and absolute references

Every cell has an address: column letter + row number (O1 = column O, row 1). A formula starts with `=`. `$` “locks” part of a reference so it doesn’t shift when copied: `$C$18` always points to C18, while `C18` would move. You will use formulas like these to audit the output in Section 17.3.

11.2 How StockDatabase.xlsx works

StockDatabase.xlsx							Data
O2 <code>=EOMONTH(03,VLOOKUP(04,R1:T3,3,FALSE))</code>							
	O	P	Q	R	S	T	
1	IBM			DAILY	0	-61	
2	8/31/21			WEEKLY	1	-121	
3	9/2/26			MONTHLY	2	-241	
4	DAILY						

The input block: O1 ticker, O2 start date, O3 end date, O4 timeframe dropdown; R1:T3 is the lookup table.

Cell	Content	What it does
O1	IBM (typed)	The ticker. This is the cell xlwings changes.
O3	<code>=TODAY()</code>	End date = today.
O4	DAILY (dropdown)	A <i>data validation</i> list restricted to R1:R3 (DAILY / WEEKLY / MONTHLY). xlwings changes this too.
R1:T3	Lookup table	Column S = STOCKHISTORY interval code (0/1/2). Column T = months of history to go back (−61 / −121 / −241).
O2	<code>=EOMONTH(03, VLOOKUP(04,R1:T3,3,FALSE))</code>	Start date. VLOOKUP finds DAILY in R1:R3 and returns column 3 (−61); EOMONTH gives the month-end 61 months before today ≈ 5 years.
A1	<code>=STOCKHISTORY(O1,O2,O3, VLOOKUP(04,R1:S3,2,FALSE), 1, 0,2,3,4,1,5)</code>	Downloads and <i>spills</i> the price table.

11.3 STOCKHISTORY, argument by argument

#	Argument	Our value	Meaning
1	stock	O1	Ticker (or exchange-qualified, e.g. XNAS:META)
2	start_date	O2	First date
3	end_date	O3	Last date
4	interval	VLOOKUP → 0/1/2	0 = daily, 1 = weekly, 2 = monthly
5	headers	1	Show a header row (Date, Open, ...)
6-11	property0...5	0, 2, 3, 4, 1, 5	Which columns, in order: 0 Date, 2 Open, 3 High, 4 Low, 1 Close, 5 Volume

11.4 Spill ranges and #BUSY!

STOCKHISTORY is a **dynamic array** function: one formula in A1 fills (“spills”) a whole block, A1:F1260. When O1 changes, Excel contacts Microsoft’s market-data service over the internet and shows **#BUSY!** until the data arrives, typically a few seconds later. This is the single most important fact for the automation: **after changing the ticker, the program must wait**. Chapter 13 shows how.

WATCH OUT

The prices are not stored in the file permanently. Open StockDatabase.xlsx offline and you only see the last cached values. That is why a normal file reader like pandas can’t fetch META: only a *running* Excel can refresh STOCKHISTORY, and only xlwings can drive a running Excel.

11.5 .xlsx vs .xlsm, macros and VBA

.xlsx workbooks cannot contain code. **.xlsm** (“macro-enabled”) workbooks can contain **VBA** (Visual Basic for Applications), Excel’s built-in programming language. A **macro** is a VBA procedure (`Sub ... End Sub`) that can be attached to a button. This project doesn’t need macros: both ways of running it work with the normal .xlsx files.

Python from zero

12.1 What Python is

Python is a programming language: you write instructions in a text file ending in `.py` (a **script**), and a program called the **interpreter** (`python`) reads and executes them line by line. It is the dominant language in data analysis and quantitative finance because it is readable and has an enormous ecosystem of free libraries.

12.2 The Terminal

The Terminal (macOS: Applications → Utilities → Terminal, or the terminal inside VS Code / Cursor) is a text window where you type commands. Five you need:

Command	What it does
<code>cd "/Users/oscar/Desktop/TAMID Fall 2026"</code>	change d irectory: go into the project folder (quotes needed because of spaces)
<code>ls</code>	l ist the files in the current folder
<code>.venv/bin/python run_analysis.py</code>	Run a script with the project's Python
<code>.venv/bin/python</code>	Open an interactive Python session (type code, press Enter, see results; exit with <code>exit()</code>)
<code>Ctrl + C</code>	Stop a running program

12.3 Packages, pip and virtual environments

A **package** (or library) is code someone else wrote that you can import, e.g. `import pandas`. **pip** is the tool that downloads packages from PyPI, the public Python package index. A **virtual environment** is a private folder (our `.venv`) holding its own Python and its own set of packages, so this project's versions can't conflict with any other project on your Mac.



That is why every command in this guide starts with `.venv/bin/python`: it guarantees the project's own Python, with `xlwings` installed, is used. `requirements.txt` lists the packages so anyone (e.g. Haas) can recreate the environment with one command.

12.4 The libraries and what each does here

Library	In one sentence	Where we use it
xlwings	Lets Python control a running Excel.	Set Data!O1/O4, wait, read prices; create, fill and save the output workbook (Ch. 13).
pandas	Spreadsheet-like tables (DataFrames) in Python.	Holds the price table; <code>shift(1)</code> for previous close; rolling, groupby.
numpy	Fast math on arrays of numbers.	<code>np.log</code> , <code>np.exp</code> , <code>np.sqrt</code> on 1,258 values at once.
scipy	Scientific and statistical functions.	Skewness, kurtosis, normal CDF, χ^2 p-value for Jarque-Bera.
matplotlib	Charting.	Histogram and price chart pictures on the tab.

12.5 A DataFrame in 60 seconds

```
import pandas as pd, numpy as np
df = pd.DataFrame({"Close": [100, 102, 99, 101, 104, 103]}) # a one-column table
df["Prev"] = df["Close"].shift(1) # yesterday's close, moved down one row
df["LogReturn"] = np.log(df["Close"] / df["Prev"]) # whole column at once, like filling
down in Excel
print(df["LogReturn"].mean(), df["LogReturn"].std()) # 0.005912 0.024792 (the worked
example)
```

TRY IT

In Terminal: `cd` into the project, run `.venv/bin/python` , paste the four lines above, press Enter. You just reproduced Chapter 4.

xlwings, in depth

xlwings is the heart of the “run it through Excel” requirement. This chapter explains what it is, how it physically talks to Excel, every operation we use, and the two ways we run the analysis.

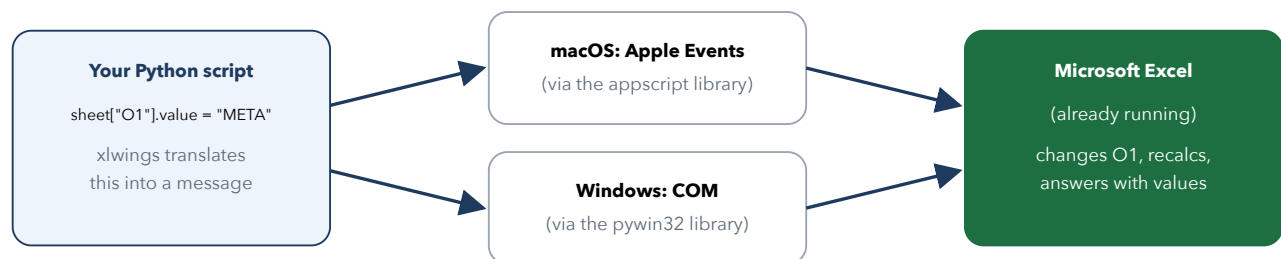
13.1 The problem xlwings solves

There are two very different ways for Python to work with Excel files:

	File libraries (openpyxl, pandas)	xlwings
Needs Excel installed?	No: reads/writes the .xlsx file directly	Yes: controls the Excel application
Sees live values?	Only values saved in the file	Yes: exactly what's on screen right now
Can recalculate / refresh STOCKHISTORY?	No	Yes
Can be triggered from Excel?	No	Yes (button, RunPython, xlwings Lite)
Speed on large files	Fast	Slower (each call is a message to Excel)

Our project uses xlwings for everything: switching the ticker, reading the Data sheet, building the analysis tab and saving the new workbook.

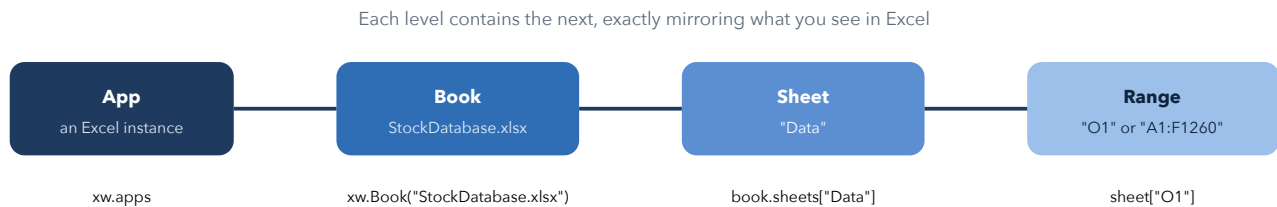
13.2 How it physically works



The operating system's inter-application messaging: the same mechanism AppleScript and VBA automation use

Every time you read or write a cell, xlwings sends a message to the Excel process and waits for a reply. That's why macOS asked the first time: “Terminal wants to control Microsoft Excel”. Apps need your permission to send each other these messages (System Settings → Privacy & Security → Automation).

13.3 The object model: App → Book → Sheet → Range



13.4 The operations, one by one

```
import xlwings as xw
import pandas as pd

book = xw.Book("StockDatabase.xlsx") # attach to the open workbook (opens it if closed)
sheet = book.sheets["Data"] # pick a sheet by name

sheet["O1"].value # READ one cell -> 'IBM'
sheet["O1"].value = "META" # WRITE one cell -> Excel starts downloading META
sheet["A1:F3"].value # READ a block -> list of rows [['Date', 'Open', ...], [...]]
sheet["O2"].formula # READ the formula ->
'EOMONTH(O3,VLOOKUP(O4,R1:T3,3,FALSE))'
sheet["A1"].expand("down").last_cell.row # how far does the table go? -> 1260

table = sheet["A1:F1260"].options(pd.DataFrame, index=False, header=True).value
# READ a block straight into a pandas DataFrame (a
"converter")
book.app.calculate() # ask Excel to recalculate
sheet["H1"].value = table.head() # WRITE a DataFrame into cells (don't save StockDatabase!)
new = xw.Book() # create a brand-new blank workbook
new.sheets[0].pictures.add(fig) # paste a matplotlib chart as a picture
new.save("my_output.xlsx") # save it
```

DEFINITION • CONVERTER / OPTIONS()

`.options(...)` tells xlwings how to translate between Excel cells and Python objects. `pd.DataFrame`, `header=True`, `index=False` means “the first row holds column names; don’t use the first column as an index.” Without it you get plain lists of lists.

13.5 Hands-on: control Excel from Python in five minutes

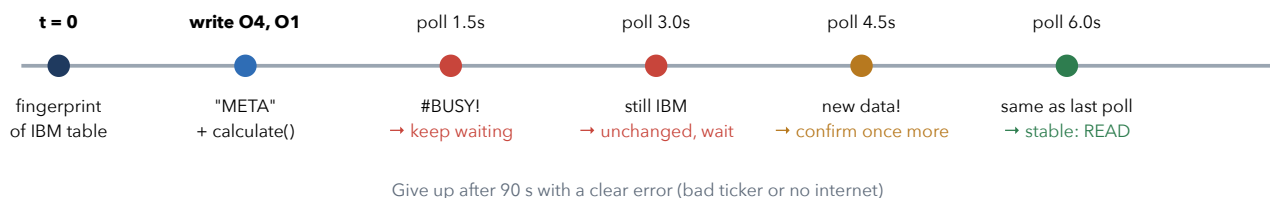
TRY IT · DO THIS NOW, WITH STOCKDATABASE.XLSX OPEN AND VISIBLE

- 1 Open Terminal and type: `cd "/Users/oscar/Desktop/TAMID Fall 2026"` then `.venv/bin/python .`
You should see `>>>` .
- 2 `import xlwings as xw` then `book = xw.Book("StockDatabase.xlsx")` . Nothing visible happens; Python is now connected.
- 3 `sheet = book.sheets["Data"]` then `sheet["O1"].value` . It prints the current ticker, e.g. `'IBM'` .
- 4 `sheet["O1"].value = "MSFT"` . **Watch Excel:** O1 changes, the table shows #BUSY!, then Microsoft's prices appear.
- 5 `sheet["E2"].value` prints MSFT's first close. `sheet["A1"].expand("down").last_cell.row` prints the row count.
- 6 `sheet["O4"].value = "WEEKLY"` . The table shrinks to weekly bars over ~10 years.
- 7 Put it back: `sheet["O4"].value = "DAILY"` and `sheet["O1"].value = "IBM"` , then `exit()` . Close StockDatabase **without saving**.

You have just done by hand exactly what `run_analysis.py` does automatically.

13.6 The waiting problem, and how we solved it

After step 4 above, if Python read the table *immediately* it would get #BUSY! or, worse, the **old** ticker's prices. So `excel_source.py` takes a **fingerprint** of the table (first date, first close, number of rows, last close) before changing the ticker, then checks every 1.5 seconds:



Two conditions must both be true before reading: (1) the fingerprint differs from the old ticker's, and (2) it is identical in two consecutive polls, so the spill has finished filling in.

13.7 The two ways we run the analysis

“xlwings” is a family of tools. This project uses two of them, and both run **the same analysis code**, the file `xlwings_lite/main.py`. That is what guarantees the two ways always produce identical tabs.

	A · <code>run_analysis.py</code> in VS Code	B · xlwings Lite inside Excel
How you start it	► Run in VS Code, or a command in the terminal	The green Run button in the xlwings Lite pane
Where Python runs	On your Mac (the project's <code>.venv</code>)	Inside Excel, sandboxed (WebAssembly)
What it analyses	The ticker/timeframe in O1/O4, or the tickers you type	The ticker/timeframe in O1/O4
Changes O1/O4 and waits for STOCKHISTORY?	Yes	No: you choose, wait, then click Run
Result	A new one-tab workbook per ticker, saved as <code>output/Analysis_<TICKER>_<TIMEFRAME>_<date>.xlsx</code>	A new tab <code><TICKER>_<TIMEFRAME>_<date></code> inside StockDatabase
Several tickers per run	Yes	No, one per click
Install	Python + packages in <code>.venv</code> (done)	Nothing: an Excel add-in; paste the script once

OTHER XLWINGS FLAVOURS WE DON'T USE

RunPython lets a VBA macro in a macro-enabled `.xlsm` file launch Python from a button (it needs `xlwings runpython install` on a Mac). **xlwings Server** runs Python on a web server. Both work, but two ways that run the same code are simpler to maintain.

Way A: VS Code remote-controls Excel

`run_analysis.py` starts outside Excel and drives it, exactly like the hands-on exercise in 13.5. For each ticker it sets O1/O4 (skipped if they already show that ticker), waits for the refresh (13.6), reads the Data sheet with **the same function xlwings Lite uses**, builds the tab in a brand-new workbook, deletes the blank starter sheet, and saves. Because desktop xlwings controls the whole Excel application, it can create and save files, which xlwings Lite cannot.

Way B: xlwings Lite, Python inside the workbook

xlwings Lite is a free Excel add-in (the side panel titled “xlwings Lite”). It contains a full Python compiled to **WebAssembly** by the **Pyodide** project, running inside Excel’s built-in web engine. Nothing is installed on your Mac, and the code is saved inside the workbook.

WHY THE PANEL LOOKS EMPTY AT FIRST

A workbook opened with xlwings Lite for the first time shows the add-in’s **demo** code (“Run Hello World”). xlwings Lite can’t read files from your disk, so `xlwings_lite/main.py` has to be pasted into the pane once. And when you use Way A, nothing shows in the pane at all: VS Code drives Excel from outside.

INSTALLING THE SCRIPT IN XLWINGS LITE (ONCE)

- 1 Open `StockDatabase.xlsx` → **Home** → **Add-ins** → **xlwings Lite**.
- 2 Click inside the `main.py` editor, select all (⌘A) and delete the demo code.
- 3 Open `xlwings_lite/main.py` in VS Code, copy everything, and paste it into the pane.
- 4 Open `requirements.txt` from the pane’s file menu (📁 ▼). Keep the `xlwings` line and add `pandas`, `numpy`, `scipy`, `matplotlib`, one per line. Lite installs them and restarts.
- 5 The green button now shows **lognormal_current_ticker**.

USING IT

- 1 On the Data sheet set O1 and O4, and wait until the table has refreshed (no #BUSY!).
- 2 Click the green ► **lognormal_current...** button (not the ▼ arrow).
- 3 A tab such as `META_DAILY_2026-09-14` appears after the existing tabs. Running it again the same day replaces it.
- 4 Errors appear in the pane’s **OUTPUT** area.

HOW AN XLWINGS LITE SCRIPT IS WRITTEN

```
from xlwings import script

@script
def lognormal_current_ticker(book: xw.Book):
    ticker, timeframe, raw = read_data_sheet(book)
    sh = write_analysis_sheet(book, ticker, timeframe, raw)
    sh.activate()
```

The `@script` decorator registers the function in the Run dropdown, and xlwings Lite passes in the workbook it is running in. Behind the scenes the script reads what it needs, computes in Python, and sends a list of changes

back to Excel **when the function finishes**. That design explains its limits:

- **It can't wait for STOCKHISTORY.** Changing O1 and reading the new prices would need Excel to download data *during* the script, so you choose the ticker before clicking Run.
- **It can't create or save separate files.** It only edits the workbook it lives in. To get a separate file, use Way A, or right-click the tab → **Move or Copy...** → **(new book)** → Save As.
- **Some desktop features aren't supported.** If the script asks for one (for example setting column widths), Excel stops applying the remaining changes and the pane shows an error like `TypeError: globalThis.callbacks[action.func] is not a function`. A Python `try/except` can't catch it, because the failure happens in Excel after Python has finished. The shared script sticks to operations both versions support.

13.8 Other xlwings features worth knowing

- **Custom functions:** write a Python function and call it from a cell like `=LOGRET(E2,E3)` (`@func` in xlwings Lite; `@xw.func` in classic xlwings on Windows).
- `xw.App(visible=False)` runs a hidden Excel for batch jobs.
- `book.macro("Name")()` runs a VBA macro from Python.

KEY IDEA

xlwings is a remote control for a running Excel. Desktop xlwings (Way A) controls the whole application: it can switch tickers, wait, and create files. xlwings Lite (Way B) lives inside one workbook: simpler, nothing to install, but limited to that workbook.

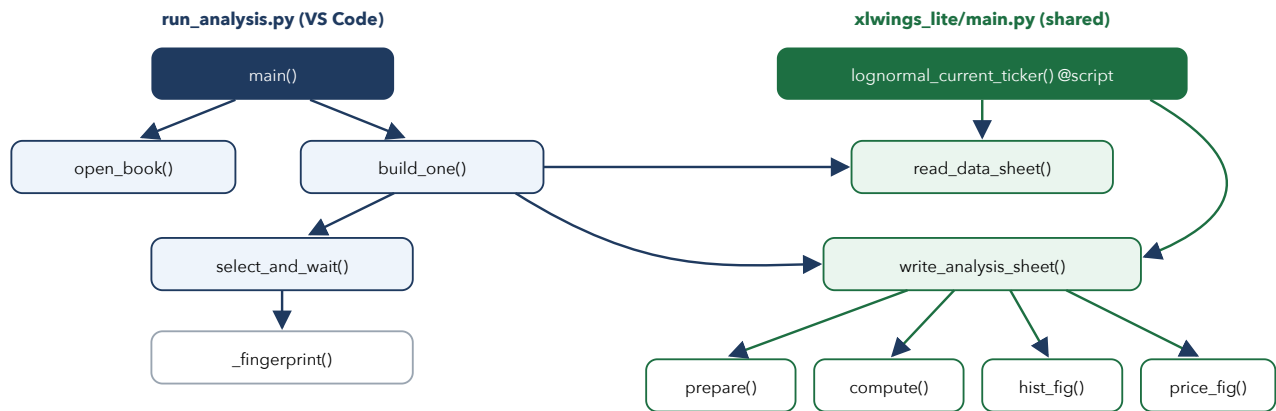
The code, function by function

The whole project is two Python files. `xlwings_lite/main.py` contains the analysis and the layout of the tab; `run_analysis.py` is a thin driver that lets VS Code reuse it. This chapter explains every function.

14.1 Project map

File / folder	Role
<code>StockDatabase.xlsx</code>	Data source: O1/O4 choose the stock; STOCKHISTORY fills A:F. Never saved by the code.
<code>xlwings_lite/main.py</code>	The analysis and tab layout. Pasted into xlwings Lite and imported by <code>run_analysis.py</code> .
<code>run_analysis.py</code>	VS Code driver: select ticker → wait → read → new one-tab workbook → save.
<code>output/</code>	<code>Analysis_<TICKER>_<TIMEFRAME>_<YYYY-MM-DD>.xlsx</code> , one per ticker per run date.
<code>docs/</code>	This guide and the technical report.
<code>.venv/</code> , <code>requirements.txt</code>	The project's private Python and its package list (xlwings, pandas, numpy, scipy, matplotlib).
<code>README.md</code>	One-page quick reference.

14.2 How the functions call each other



Blue arrows: VS Code path · Green arrows: xlwings Lite path · both end in the same `write_analysis_sheet()`

14.3 `xlwings_lite/main.py` : the shared analysis

`read_data_sheet(book)`

Does: reads the ticker (O1), the timeframe (O4) and the STOCKHISTORY price table from the Data sheet.

Returns: `(ticker, timeframe, raw)`, where `raw` is a pandas DataFrame with Date, Open, High, Low, Close, Volume.

```
sh = book.sheets["Data"]
ticker = str(sh["01"].value).strip().upper()
timeframe = str(sh["04"].value).strip().upper()
raw = sh["A1"].expand("down").resize(column_size=6).options(pd.DataFrame, index=False,
header=True).value
```

`expand("down")` finds how far the table goes (1,260 rows for 5 years of daily bars); `resize(column_size=6)` widens it to columns A:F; the `options(pd.DataFrame ...)` converter turns the block into a table with the first row as column names.

`prepare(raw)`

Does: cleans the table and adds the return columns. **Returns:** a DataFrame with `LogReturn` and `Z` added.

- Normalises column names ("close" → "Close").
- If dates arrive as Excel serial numbers (days since 1899-12-30), converts them to real dates.
- Forces Close to numbers, drops blanks and non-positive prices, sorts by date, removes duplicate dates.
- `LogReturn = ln(Close / previous Close)` (Ch. 3); the first row has none.
- `Z = (LogReturn - mean) / std` (Ch. 5.2).

`compute(df, ticker, timeframe)`

Does: all the statistics. **Returns:** a dictionary with `mu`, `sd`, `summary` (the 13 rows of the statistics block, each with its Excel number format) and `bounds` (the three sigma rows).

```
mu, sd = r.mean(), r.std(ddof=1)                                Ch. 4
skew, kurt = st.skew(r, bias=False), st.kurtosis(r, bias=False) Ch. 8 (= Excel SKEW, KURT)
jb_p = st.chi2.sf(n / 6 * (skew**2 + kurt**2 / 4), 2)           Ch. 8.4
for k in (1, 2, 3):
    lo, hi = mu - k*sd, mu + k*sd                                log band
    normal = 2*st.norm.cdf(k) - 1                                68.27 / 95.45 / 99.73%
    moves = np.exp(lo) - 1, np.exp(hi) - 1                       % moves (Ch. 6)
    prices = p0*np.exp(lo), p0*np.exp(hi)                       price band from last close
    empirical = ((r >= lo) & (r <= hi)).mean()                   share of real days inside
```

Every value is converted to a plain Python `float / int`, because xlwings Lite has to send the numbers to Excel as JSON and can't send numpy types. The annualizing factor comes from `PERIODS_PER_YEAR`: 252 daily, 52 weekly, 12 monthly.

`hist_fig(df, mu, sd, ticker, timeframe)` and `price_fig(df, ticker)`

Do: draw the two matplotlib charts. `hist_fig` is a 60-bin density histogram of the log returns (clipped at $\pm 4.5\sigma$ so a few extreme days don't squash the picture), the fitted normal curve, and dashed $\pm 1\sigma$ (green), $\pm 2\sigma$ (amber), $\pm 3\sigma$ (red) lines, with a percentage x-axis. `price_fig` is the close price over time. **Return:** matplotlib figures.

`write_analysis_sheet(book, ticker, timeframe, raw)`

Does: builds the whole tab. **Returns:** the new sheet.

- 1 Calls `prepare` and `compute` .
- 2 Names the tab `<TICKER>_<TIMEFRAME>_<YYYY-MM-DD>` (cut to Excel's 31-character limit). If a tab with that name exists, it's deleted first, so a same-day rerun replaces it.
- 3 A1 title (bold, 14 pt), A2 "Generated ... from the Data sheet", A3 the suggested file name.
- 4 A4:B17 statistics, each value with its own number format (dates, #,##0, \$, %, decimals).
- 5 A19:G22 the sigma table: band, normal confidence, lower/upper % move, lower/upper price, empirical coverage.
- 6 I1:L1260 the data: Date, Close, Log Return, Z-Score, written in one block (much faster than cell by cell).
- 7 The histogram at A25 and the price chart at A50, added with `sheet.pictures.add(fig, anchor=...)` .

`lognormal_current_ticker(book)`

Does: the xlwings Lite entry point (the name in the Run dropdown). It calls `read_data_sheet` , then `write_analysis_sheet` , then activates the new tab so you land on it.

14.4 `run_analysis.py` : the VS Code driver

`main()`

Does: reads the command line, decides what to analyse, runs each ticker, and restores StockDatabase.

```
tickers = [t.upper() for t in args.tickers] or [01]      no tickers typed → the one selected in Excel
timeframe = args.timeframe or 04                       no -t → the timeframe selected in Excel
try:
    for t in tickers:
        out = build_one(source, t, timeframe)
finally:
    04, 01 = original values                            runs even if a ticker fails
```

So ► Run (no arguments) analyses whatever StockDatabase shows, and `run_analysis.py META ORCL -t WEEKLY` analyses a list. Before anything else it sets `MPLBACKEND=Agg` so charts are drawn off-screen, and adds the `xlwings_lite` folder to Python's path so `import main as lite` loads the shared file.

`open_book(path)`

Does: looks through every open Excel window for `StockDatabase.xlsx` and attaches to it; opens it only if it isn't open. This reuses your window instead of opening a second copy.

`select_and_wait(book, ticker, timeframe, timeout=90)`

Does: makes StockDatabase show the requested ticker and waits until the data is really there.

- 1 If O1/O4 already show that ticker and timeframe, return immediately (that's the ► Run case).
- 2 Take a fingerprint of the current table, write the timeframe into O4 and the ticker into O1, ask Excel to calculate.
- 3 Every 1.5 s take a new fingerprint. Stop when it's **different from before** (new stock) **and identical to the previous check** (finished loading).
- 4 After 90 s without data, raise a clear error (bad ticker or no internet).

`_fingerprint(sh)`

Does: returns (first date, first close, last row, last close) of the price table, or `None` while it isn't valid yet (`#BUSY!`, wrong header, missing numbers). Reads are wrapped in `try/except` because on macOS reading a range that STOCKHISTORY is still filling can raise an error instead of returning blanks.

`build_one(source, ticker, timeframe)`

Does: produces and saves one workbook.

```

select_and_wait(source, ticker, timeframe)
ticker, timeframe, raw = lite.read_data_sheet(source)
out = OUTPUT / f"Analysis_{ticker}_{timeframe}_{today}.xlsx"
close an already-open copy of that file
book = xw.Book()
blank = book.sheets[0].name
lite.write_analysis_sheet(book, ticker, timeframe, raw)
book.sheets[blank].delete()
book.save(str(out))

```

same reader as xlwings Lite

new blank workbook (Sheet1)

same layout as xlwings Lite

one tab left

WHY DELETE THE BLANK SHEET BY NAME

On macOS, xlwings refers to `book.sheets[0]` by *position*. Adding the analysis tab moves it into position 0, so deleting “sheet 0” would delete the new tab and leave the empty Sheet1. That happened in testing; remembering the name first fixed it.

14.5 Design decisions

Decision	Why
One shared analysis file	VS Code and xlwings Lite can never drift apart: same reader, same statistics, same layout.
Values, not Excel formulas	Python does the math once; the tab is a clean snapshot that copies anywhere. You can still audit any number with a formula (Section 17.3).
One workbook per ticker, dated	Easy to share and archive; the date shows which data was used. Same ticker/timeframe/day overwrites; a new day makes a new file.
O1/O4 restored in finally	StockDatabase goes back to what you had selected, even after an error. It is never saved.
Only operations xlwings Lite supports	Unsupported calls (like column widths) break the Lite run after Python finishes.

PART IV

Operating manual

Install, run, change tickers, find your way around the output workbook, read and present the results, and fix problems.

| Chapters 15–19 · Checklists · Real previews of every workbook · Meeting talking points · Troubleshooting table

Installation and first run

On Oscar's Mac everything is already installed. The checklist is here so you understand each piece, and so Haas or anyone else can set it up.

15.1 One-time checklist

- ❑ **Excel for Microsoft 365** (STOCKHISTORY needs it) and internet access.
- ❑ **Python 3.10+**: `python3 --version` in Terminal.
- ❑ **VS Code** with the Python extension; open the folder `TAMID Fall 2026`.
- ❑ **Create the environment** (in VS Code's terminal): `python3 -m venv .venv`
- ❑ **Install the packages**: `.venv/bin/pip install -r requirements.txt`
- ❑ **Select the interpreter**: `⌘⇧P` → *Python: Select Interpreter* → `.venv`.
- ❑ **Allow automation**: on the first run click **OK** on "Code wants to control Microsoft Excel" (or Terminal).
- ❑ **xlwings Lite**: paste `xlwings_lite/main.py` and the requirements into the pane (Section 13.7).

15.2 First run

- 1 Open `StockDatabase.xlsx` and let it finish loading.
- 2 In VS Code open `run_analysis.py`, then open the terminal (`⌘``).
- 3 Run `.venv/bin/python run_analysis.py META ORCL`.
- 4 Watch Excel: O1 switches to META, the table refreshes, a new workbook appears and is saved; then ORCL; then O1/O4 go back.
- 5 Both files are in `output/` and stay open in Excel.

VS Code terminal – TAMID Fall 2026

```
oscar@Oscars-MacBook-Pro TAMID Fall 2026 % .venv/bin/python run_analysis.py META ORCL
META DAILY: loading...
saved output/Analysis_META_DAILY_2026-09-14.xlsx
ORCL DAILY: loading...
saved output/Analysis_ORCL_DAILY_2026-09-14.xlsx
```

READING THE COMMAND

`.venv/bin/python` = the project's Python · `run_analysis.py` = the script · `META ORCL` = tickers (optional) · `-t WEEKLY` = timeframe (optional; default is whatever O4 shows).

Everyday use

16.1 Option 1: choose in Excel, click ► Run in VS Code

The simplest way to use Excel as the menu:

- 1 In StockDatabase's Data sheet, type the ticker in O1 and pick the timeframe in O4.
- 2 In VS Code, with `run_analysis.py` open, click ► **Run** (top right).
- 3 You get `output/Analysis_<TICKER>_<TIMEFRAME>_<date>.xlsx`.

With no tickers given, the script analyses exactly what O1/O4 show. You don't even need to wait for the refresh: it checks the data is loaded before reading.

16.2 Option 2: several tickers from the terminal

Command	Result
<code>.venv/bin/python run_analysis.py</code>	Same as ► Run: the ticker and timeframe selected in Excel
<code>.venv/bin/python run_analysis.py META ORCL</code>	Oscar's assignment: two files, timeframe from O4
<code>.venv/bin/python run_analysis.py AAPL MSFT -t DAILY</code>	Haas's assignment
<code>.venv/bin/python run_analysis.py META ORCL -t WEEKLY</code>	Weekly bars (~10 years)
<code>.venv/bin/python run_analysis.py SPY -t MONTHLY</code>	Market benchmark, monthly bars (~20 years)
<code>.venv/bin/python run_analysis.py --help</code>	Lists the options

16.3 Option 3: xlwings Lite, without leaving Excel

- 1 Set O1 and O4 and wait for the table to refresh.
- 2 xlwings Lite pane → ► **lognormal_current_ticker**.
- 3 The tab `<TICKER>_<TIMEFRAME>_<date>` appears inside StockDatabase.
- 4 To turn it into its own file: right-click the tab → **Move or Copy...** → (new book), tick **Create a copy** → OK → **Save As** in `output/` using the name written in cell A3.

16.4 Files, names and reruns

Situation	What happens
Same ticker, same timeframe, same day	The file (or Lite tab) is replaced with fresh data
Same ticker on a later day	A new dated file/tab; the old one is kept
Output file already open in Excel	The script closes it first, then saves the new one
StockDatabase after a VS Code run	O1/O4 back to what they were; the file isn't saved

WATCH OUT

The data window rolls forward every day (today minus 61 months for daily). Numbers presented today will differ slightly next week; the run date in the file name tells you which data you used.

Tour of the output workbook

Each file, e.g. `output/Analysis_META_DAILY_2026-09-14.xlsx`, has exactly one tab with the same layout whether it came from VS Code or xlwings Lite. The previews below are drawn from the real META file.

17.1 Header and statistics (A1:B17)

Analysis_META_DAILY_2026-09-14.xlsx		META_DAILY_2026-09-14
B12	f_x	2.85%
	A	B
1	META: lognormal return distribution (daily)	
2	Generated 2026-09-14 17:12 by xlwings Lite from the Data sheet	
3	File name: Analysis_META_DAILY_2026-09-14.xlsx	
4	Statistic	Value
5	Ticker	META
6	Timeframe	DAILY
7	First date	8/31/2021
8	Last date	9/4/2026
9	Return observations	1,258
10	Last close	\$616.77
11	Mean log return (μ)	0.039%
12	Std. deviation (σ)	2.85%
13	Annualized drift	9.73%
14	Annualized volatility	45.25%
15	Skewness	-1.096
16	Excess kurtosis	23.414
17	Jarque-Bera p-value	0.0000
Analysis_META_DAILY_2026-09-14.xlsx: title, run line, file name and the statistics block.		

Row	Meaning · chapter
Ticker, Timeframe, First/Last date	What was analysed and the data window · Ch. 2
Return observations	n = number of returns (prices – 1) · Ch. 4
Last close	P_0 , the starting point for the price bands · Ch. 6
Mean log return (μ), Std. deviation (σ)	Centre and spread per bar · Ch. 4
Annualized drift / volatility	$\mu \times 252$ and $\sigma \times \sqrt{252}$ for daily data · Ch. 7
Skewness, Excess kurtosis	Lopsidedness and fat tails (normal = 0) · Ch. 8
Jarque-Bera p-value	Below 0.05 = returns are not normal · Ch. 8.4

17.2 The sigma table (A19:G22)

Analysis_META_DAILY_2026-09-14.xlsx							META_DAILY_2026-09-14
G20 fx 81.96%							
	A	B	C	D	E	F	G
19	Band	Normal confidence	Lower % move	Upper % move	Lower price	Upper price	Empirical coverage
20	$\pm 1\sigma$	68.27%	-2.77%	2.93%	\$599.67	\$634.85	81.96%
21	$\pm 2\sigma$	95.45%	-5.51%	5.91%	\$582.82	\$653.21	96.34%
22	$\pm 3\sigma$	99.73%	-8.16%	8.97%	\$566.44	\$672.09	98.81%
The $\pm 1\sigma$ / $\pm 2\sigma$ / $\pm 3\sigma$ table on the META analysis tab (A19:G22).							

For $\pm 1\sigma$, $\pm 2\sigma$ and $\pm 3\sigma$: the probability a normal distribution puts inside the band, the next-bar down and up moves in %, the same band as prices from the last close, and the share of real historical days that actually landed inside (Ch. 5, 6, 8.1).

17.3 The data columns (I:L)

Analysis_META_DAILY_2026-09-14.xlsx

META_DAILY_2026-09-14

K3fx0.70%

	I	J	K	L
1	Date	Close	Log Return	Z-Score
2	8/31/21	\$379.38		
3	9/1/21	\$382.05	0.70%	0.23
4	9/2/21	\$375.28	-1.79%	-0.64
5	9/3/21	\$376.26	0.26%	0.08
6	9/7/21	\$382.18	1.56%	0.53
7	9/8/21	\$377.57	-1.21%	-0.44

Columns I-L: every trading day with its close, log return and z-score.

One row per trading day: date, close, log return (empty on the first day) and z-score. Use Excel's filter on column L to list the biggest days: > 3 or < -3 shows the earnings shocks from Section 8.6.

TRY IT • AUDIT THE NUMBERS WITH YOUR OWN FORMULAS

The tab holds values computed by Python. Check them in any empty cell of the tab:

- `=STDEV(K3:K1260)` → 2.85% (matches B12)
- `=AVERAGE(K3:K1260)` → 0.039% (B11) • `=KURT(K3:K1260)` → 23.414 (B16)
- `=COUNTIFS(L3:L1260,">=1",L3:L1260,"<=1")/1258` → 81.96% (G20)

If Excel agrees with Python, you have independently verified the analysis.

17.4 The charts (A25 and A50)

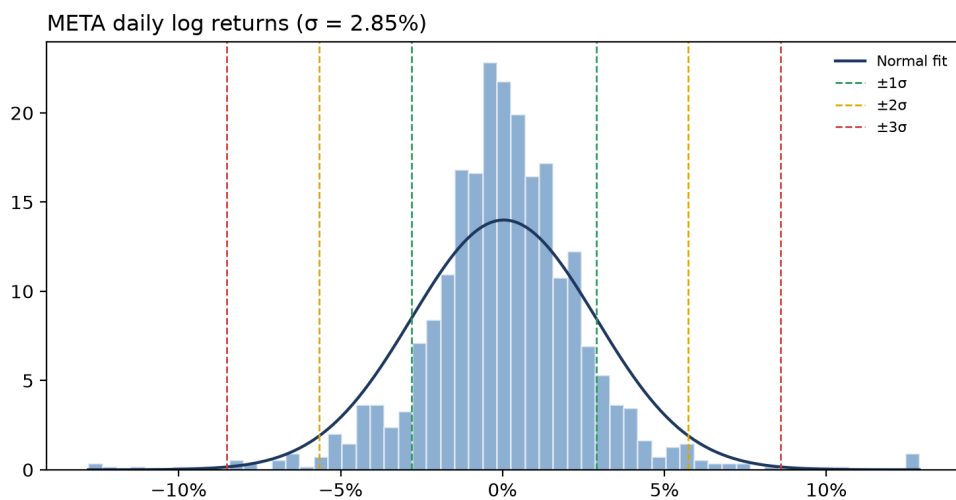


Figure 17.1 The histogram picture at A25, exactly as saved in the META file: bars = real daily log returns, curve = normal fit, dashed lines = $\pm 1\sigma/\pm 2\sigma/\pm 3\sigma$.

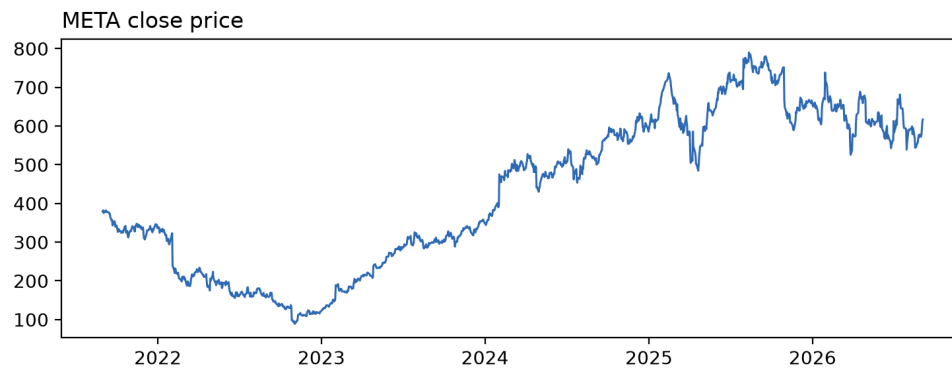


Figure 17.2 *The price chart at A50.*

Both are pictures (not native Excel charts), so they copy along with the tab and look the same on any computer.

Reading and presenting the results

18.1 META



Figure 18.1 META close and daily log returns (red = beyond $\pm 2\sigma$).

Metric	Value	Interpretation
Daily σ / annualized	2.85% / 45.3%	High-volatility mega-cap; $\sim 2.5\times$ the S&P 500
Annualized drift	9.7%	$\ln(616.77/379.38)$ over 5 years; not statistically reliable
Skew	-1.10	Biggest shocks were to the downside (2022)
Excess kurtosis	23.4	Extremely fat tails; mostly five earnings days
Inside $\pm 1\sigma$ / $\pm 2\sigma$ / $\pm 3\sigma$	82.0 / 96.3 / 98.8%	vs 68.3 / 95.4 / 99.7: fat tails
Next-day 2σ range	\$582.82 - \$653.21	"Normal day" range for expectations and stops
1-month 1σ range	\$545.65 - \$708.56	$\sqrt{21} \times$ daily σ

18.2 ORCL



Figure 18.2 ORCL close and daily log returns.

Metric	Value	Interpretation
Daily σ / annualized	2.69% / 42.6%	Similar to META over 5 years, but very uneven: ~30% until 2024, ~59% in 2025–26
Skew	+1.25	Dominated by the +36% jump of Sep 10, 2025
Excess kurtosis	17.9	Very fat tails
Inside $\pm 1\sigma$ / $\pm 2\sigma$ / $\pm 3\sigma$	81.4 / 95.4 / 98.4%	Same fat-tail signature; 20 days beyond 3σ
Next-day 2σ range	\$150.54 – \$167.62	
1-month 1σ range	\$141.75 – \$181.32	

WATCH OUT • THE SINGLE- Σ PROBLEM FOR ORCL

ORCL's 5-year σ (43%) averages a calm 30% regime and a stormy 59% regime. Its current risk is probably understated by the 5-year number. Say this in the meeting; it shows you understand volatility clustering (Figure 8.4).

18.3 A 90-second presentation script

“I pulled five years of daily closes for META and ORCL from the StockDatabase workbook into Python with xlwings, computed log returns, and wrote the results into a one-tab Excel workbook per ticker.

Both are high-volatility names: META’s daily standard deviation is 2.85%, about 45% annualized; ORCL is 2.69%, about 43%. So a one-sigma day is roughly $\pm 2.8\%$ for META, which is \$600 to \$635 from the last close, and a two-sigma day is about $\pm 5.7\%$.

The interesting part is that the normal model doesn’t fit. About 82% of days are inside one sigma instead of 68%, and we had 15 META days and 20 ORCL days beyond three sigma when the model expects about three. Excess kurtosis is over 17 for both and Jarque-Bera rejects normality. Those tail days are almost all earnings: META’s -26% in February 2022, ORCL’s $+36\%$ last September.

So I’d use the two-sigma band as the normal-day range, but size positions and set risk limits assuming three-sigma-plus moves happen a few times a year, especially around earnings. Next steps would be a rolling or EWMA volatility, since ORCL’s vol has roughly doubled since 2024, and comparing with option-implied vol.”

18.4 Questions you may be asked, and answers

Why log returns instead of simple returns?

They add over time, they’re symmetric, they keep prices positive, and they’re the standard assumption in Black-Scholes and volatility quoting (Ch. 3).

Why are your % bounds not symmetric?

The band is symmetric in log space; converting with $e^x - 1$ makes the upside larger. That is the lognormal property (Ch. 6.3).

Is 3σ the 99% bound?

3σ is 99.73%. An exact 99% band is $\pm 2.576\sigma$. We labelled the true probability for each band (Ch. 5.3).

How do you annualize?

Multiply daily σ by $\sqrt{252}$, because variances of independent returns add (Ch. 7).

How do you know the returns aren’t normal?

Four ways: empirical coverage, excess kurtosis 23/18, Jarque-Bera $p \approx 0$, and the S-shaped Q-Q plot (Ch. 8).

How confident are you in the drift?

Not at all: its standard error is larger than the estimate. The analysis is about ranges, not direction (Ch. 4.5).

Why STDEV and not STDEV.P?

It is a sample, so $n - 1$ (Bessel’s correction).

How does the Python actually get the data?

xlwings sends Apple Events to the running Excel, sets the ticker cell, waits until the STOCKHISTORY spill has refreshed and is stable, then reads the range into a pandas DataFrame (Ch. 13).

What would you improve?

Time-varying volatility (EWMA/GARCH), fat-tailed distribution (Student-t), excluding or flagging earnings days, dividend-adjusted data, comparing with implied volatility (Ch. 10).

Troubleshooting

Symptom / message	Cause	Fix
No such file / command not found	Terminal not in the project folder	<code>cd "/Users/oscar/Desktop/TAMID Fall 2026"</code>
<code>ModuleNotFoundError: xlwings</code>	System Python used, not .venv	Start with <code>.venv/bin/python</code> , or select the .venv interpreter in VS Code
<code>OSERROR -1743 / not authorized</code>	macOS automation permission denied	System Settings → Privacy & Security → Automation → allow Excel for VS Code / Terminal
<code>STOCKHISTORY returned no data ... in 90s</code>	Wrong ticker, no internet, not signed in to Microsoft 365	Check Data!A1 in Excel; try <code>XNAS:META</code> ; check connection
Data!A1 shows <code>#BLOCKED!</code> / <code>#CONNECT!</code>	Connected data disabled or offline	Sign in to Excel; allow connected experiences
Excel asks to "Grant File Access"	Excel for Mac sandbox	Click Select... and grant the project folder
An empty Book1/Book2 with only Sheet1 is left open	A run stopped halfway	Close it without saving and run again
StockDatabase still shows another ticker	The run was force-stopped before restoring	Set O1/O4 back by hand; don't save
xlwings Lite shows only "Hello World"	Demo code still loaded	Paste <code>xlwings_lite/main.py</code> (Section 13.7)
Lite: <code>ModuleNotFoundError: scipy</code>	Pane's requirements.txt incomplete	Add pandas, numpy, scipy, matplotlib; let it restart
Lite: <code>TypeError: globalThis.callbacks[action.func] is not a function</code>	The script used an operation xlwings Lite doesn't support	Use the unmodified <code>xlwings_lite/main.py</code> ; remove the unsupported call
Lite tab appears but no charts	Same as above: Excel stopped before the pictures	Same fix, then click Run again
Lite and VS Code results differ	Different script versions, or data refreshed in between	Re-paste <code>main.py</code> into the pane; compare files from the same day

Formula sheet

Quantity	Formula	Excel
Simple return	$R = P_t/P_{t-1} - 1$	=E3/E2-1
Log return	$r = \ln(P_t/P_{t-1})$	=LN(E3/E2)
Log $\leftrightarrow$ simple	$R = e^r - 1, r = \ln(1+R)$	=EXP(r)-1
Mean	$\mu = \sum r / n$	=AVERAGE(r)
Standard deviation	$\sigma = \sqrt{[\sum (r-\mu)^2 / (n-1)]}$	=STDEV(r)
z-score	$z = (r - \mu) / \sigma$	=(G3-μ)/σ
Normal coverage	$P(Z \leq k) = 2\Phi(k) - 1$	=2*NORMSDIST(k)-1
Log band	$\mu \pm k\sigma$	
% move band	$e^{\mu \pm k\sigma} - 1$	=EXP(μ-k*σ)-1
Price band	$P_0 \cdot e^{\mu \pm k\sigma}$	=P0*EXP(μ+k*σ)
h-period price band	$P_0 \cdot e^{\mu h \pm k\sigma\sqrt{h}}$	=P0*EXP(μ*h+k*σ*SQRT(h))
Annualized vol / drift	$\sigma\sqrt{N}, \mu N$ ($N = 252/52/12$)	=σ*SQRT(252)
Empirical coverage	share of r in $[\mu - k\sigma, \mu + k\sigma]$	=COUNTIFS(r,">="&lo,r,"<="&hi)/n
Skewness	$n/((n-1)(n-2)) \sum z^3$	=SKEW(r)
Excess kurtosis	sample-corrected $\sum z^4 - 3$	=KURT(r)
Jarque-Bera	$n/6 (S^2 + K^2/4)$	=n/6*(S^2+K^2/4)
JB p-value	χ^2_2 survival	=CHIDIST(JB,2)
Normal-implied percentile	$\mu + \Phi^{-1}(p)\sigma$	=μ+NORMSINV(p)*σ
Parametric VaR (1-day, c)	$V \cdot (1 - e^{\mu - z_c\sigma})$	$z_{95\%} = 1.645, z_{99\%} = 2.326$
Options expected move	$S \cdot IV \cdot \sqrt{(T/365)}$	
Two-asset portfolio vol	$\sqrt{(w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\rho\sigma_1\sigma_2)}$	

Excel ↔ Python mapping

Task	Excel	pandas / numpy / scipy
Previous row	E2 next to E3	<code>s.shift(1)</code>
Natural log / exp / sqrt	LN / EXP / SQRT	<code>np.log</code> / <code>np.exp</code> / <code>np.sqrt</code>
Average / sample std	AVERAGE / STDEV	<code>s.mean()</code> / <code>s.std(ddof=1)</code>
Median / percentile	MEDIAN / PERCENTILE	<code>s.median()</code> / <code>s.quantile(p)</code>
Skew / kurtosis	SKEW / KURT	<code>st.skew(s, bias=False)</code> / <code>st.kurtosis(s, bias=False)</code>
Normal CDF / inverse	NORMSDIST / NORMSINV	<code>st.norm.cdf</code> / <code>st.norm.ppf</code>
Count with conditions	COUNTIFS	<code>((s >= lo) & (s <= hi)).sum()</code>
Rolling 63-day σ	fill-down STDEV on a moving range	<code>s.rolling(63).std()</code>
Group by year	PivotTable	<code>df.groupby(df.Date.dt.year)</code>

Glossary

Term	Meaning
AppleScript / Apple Events	macOS mechanism for one app to control another; what xlwings uses on Mac
Annualized volatility	σ scaled to one year: $\sigma \times \sqrt{t}$ (bars per year)
Bar	Summary of one period: Open, High, Low, Close, Volume
Bessel's correction	Dividing by $n - 1$ in the sample standard deviation
COM	Windows equivalent of Apple Events, used by xlwings on Windows
Converter	xlwings <code>.options()</code> rule for translating cells $\leftrightarrow$ Python objects
DataFrame	pandas table with named columns
Drift (μ)	Average log return per bar
Dynamic array / spill	One formula filling a block of cells (STOCKHISTORY)
Empirical coverage	Actual share of past bars inside a σ band
EWMA / GARCH	Models where volatility changes over time
Excess kurtosis	Tail heaviness relative to normal (0 = normal)
Fat tails / leptokurtic	Extreme values more frequent than a normal distribution predicts
Geometric Brownian Motion	Continuous-time model with normal log returns; basis of Black-Scholes
Implied volatility	Volatility implied by option prices; the market's forecast
Interpreter	The program that runs Python code
Jarque-Bera	Normality test from skewness and kurtosis
Log return	$\ln(P_t/P_{t-1})$
Lognormal	Variable whose logarithm is normal; model for prices
Macro / VBA	Excel's built-in programming; stored only in .xlsm files
Normal distribution	Symmetric bell curve described by μ and σ
openpyxl	Python library that reads and writes .xlsx files without Excel
pip / PyPI	Python package installer / the public package index
Pyodide / WebAssembly	Python compiled to run inside a web engine; powers xlwings Lite
p-value	Probability of a result at least this extreme if the null hypothesis (here: normality) is true
Q-Q plot	Actual vs theoretical quantiles; a straight line means the distributions match
RunPython	xlwings VBA function that launches Python from an Excel macro
Skewness	Asymmetry of the distribution (0 = symmetric)
Standard deviation (σ)	Typical distance of observations from the mean; "volatility"
Stationarity	Statistical behaviour that doesn't change over time
STOCKHISTORY	Excel 365 function downloading historical prices
Value-at-Risk	Loss not exceeded with a given confidence over a given horizon
Virtual environment	Private, per-project Python installation (<code>.venv</code>)
Volatility clustering	Big moves follow big moves; σ varies over time
xlwings	Python library to control Excel / be called from Excel
xlwings Lite	Excel add-in running sandboxed Python inside Excel
z-score	$(r - \mu)/\sigma$: a move measured in standard deviations

Self-test quiz

If you can answer these without looking, you understand the project better than most people who will hear about it. Answers are on the next page.

1. A stock goes $\$50 \rightarrow \$60 \rightarrow \$50$. What are the two simple returns and the two log returns? Which pair sums to the true total return?
2. Why can't pandas alone read META's prices from `StockDatabase.xlsx`?
3. META's daily σ is 2.85%. What is its approximate weekly σ (5 days)? Annual σ ?
4. What fraction of days should fall outside $\pm 2\sigma$ under a normal distribution? How many of 1,258?
5. Why is META's 1σ band $-2.77\% / +2.93\%$ and not $\pm 2.85\%$?
6. What does an excess kurtosis of 23 tell you, in plain English?
7. ORCL's $\pm 2\sigma$ empirical coverage is 95.39%, almost exactly the theory. Does that mean returns are normal at 2σ ? Why or why not?
8. Jarque-Bera p-value = 0.0000. What do you conclude?
9. Name two reasons returns have fat tails.
10. What does `sheet["01"].value = "META"` physically cause on a Mac?
11. Why does `load_prices` wait for the fingerprint to be *the same twice*?
12. What is the difference between running `run_analysis.py` and running the script in xlwings Lite? Which one can create the separate workbook, and why?
13. Why does the xlwings Lite pane show "Hello World", and how do you fix it?
14. Why does `build_one` delete the blank sheet by its name instead of using `book.sheets[0]`?
15. You have \$50,000 and will risk 2% on ORCL with a stop at the 2σ daily down level (\$150.54 from \$158.78). How many shares?
16. Option IV on META is 60% while realized volatility is 45%. What might that suggest, and what could explain it?
17. Why is the 5-year σ possibly misleading for ORCL right now?
18. What command runs Haas's assignment?

Answers

1. Simple: +20%, -16.67% (sum +3.33%, wrong). Log: $\ln(1.2) = +0.1823$, $\ln(50/60) = -0.1823$ (sum 0 ✓, the true total).
2. The prices come from STOCKHISTORY, which only a running Excel can refresh. pandas reads only the values last saved in the file (IBM).
3. Weekly $\approx 2.85\% \times \sqrt{5} \approx 6.4\%$. Annual $\approx 2.85\% \times \sqrt{252} \approx 45.3\%$.
4. 4.55%; about 57 days.
5. The band is symmetric in log space ($\mu \pm \sigma$); converting with $e^x - 1$ stretches the upside (lognormal asymmetry).
6. The distribution has a much taller, narrower centre and far more extreme days than a bell curve: calm most of the time, occasionally violent.
7. No. Excess calm days (inside 1σ) and excess extreme days (beyond 3σ) happen to offset at 2σ . The 1σ and 3σ coverages, kurtosis and JB all reject normality.
8. Reject the hypothesis that returns are normally distributed ($p < 0.05$).
9. Event jumps (earnings) and volatility clustering (changing regimes).
10. xlwings sends an Apple Event to Excel; Excel writes META into O1, recalculates, and STOCKHISTORY starts downloading META's history.
11. The first changed reading might be mid-refresh; two identical consecutive readings mean the spill has finished filling.
12. `run_analysis.py` runs on your Mac and controls the whole Excel application, so it can change O1/O4, wait, create a workbook and save it. xlwings Lite runs sandboxed inside one workbook, so it can only add a tab there. Both use the same `main.py`.
13. The pane loads its demo code by default and can't read files from your disk; paste `xlwings_lite/main.py` into its `main.py` and add the requirements.
14. On macOS xlwings refers to `sheets[0]` by position. Adding the analysis tab shifts positions, so deleting "sheet 0" would delete the new tab.
15. Risk \$1,000; risk per share \$8.24; 121 shares ($\approx$ \$19,212).
16. The market expects more volatility than history, e.g. an upcoming earnings release or event; options look expensive relative to realized volatility.
17. Volatility roughly doubled from $\sim 30\%$ (2022–24) to $\sim 59\%$ (2025–26); a 5-year average understates the current regime.
18. `.venv/bin/python run_analysis.py AAPL MSFT -t DAILY`

xlwings cheat sheet

```
import xlwings as xw

xw.apps                                # running Excel instances
xw.books                                # open workbooks in the active app
book = xw.Book("file.xlsx")             # attach or open      book = xw.Book()  -> new blank workbook
book = xw.Book.caller()                 # inside RunPython: the workbook that called
sheet = book.sheets["Data"]             # or book.sheets[0];   book.sheets.add("New")
rng = sheet["A1"]                       # or sheet.range("A1:F10"), sheet.range((row, col), (row2, col2))

rng.value                               # read (scalar, list, or list of lists)
rng.value = 42                          # write; also lists, NumPy arrays, DataFrames
rng.formula = "=SUM(B2:B9)"             # write a formula
rng.expand("down")                      # "table" | "down" | "right": grow to the contiguous block
rng.options(pd.DataFrame, header=True, index=False).value # read into DataFrame
rng.number_format = "0.00%"            # formatting
sheet.pictures.add(fig, name="Chart", update=True) # matplotlib figure

book.app.calculate()                   # recalc      book.save("path.xlsx")      book.close()
book.macro("SampleCall")()             # run a VBA macro
app = xw.App(visible=False)            # hidden Excel for batch work; app.quit()

# Terminal:  xlwings quickstart NAME --standalone  xlwings runpython install  xlwings --help
# Lite:      from xlwings import script, func      @script def main(book: xw.Book): ...
```

All figures in this guide come from the run of September 14, 2026 (data through September 4, 2026) and appear in the output workbooks. This is educational material, not investment advice.